



ASSIGNMENT 1

OOP Programming



Module Name: Programming Techniques 2,
Module Code: BCNS2101C,
Student Name: Sungkur Lagnesh, Kessoonath Surakshit,
Student ID: 2504_01292, 2504_28537
Course Name: BSC (Hons) Computer Science with Network security
Cohort: BCNS/25A/FT,
Lecturer: Mr. Ajit Kumar Gopee
Date Of Submission: 5 June 2026

Certificate of Originality

This is to confirm that We, **Sungkur Lagnesh** (Student ID: **2504_01292**) and **Kessoonath Surakshit** (Student ID: **2504_28537**) are the authors of this assignment, "**Assignment 1**," submitted for **BCNS2101C Programming Techniques 2**. we have not reproduced any part of this work from other sources, unless clearly attributed and referenced.

- All third-party ideas, data, or text used have been cited correctly in line with academic standards.
- We assert that this submission meets all University requirements regarding plagiarism and academic integrity.

© 2025 **Sungkur Lagnesh** and **Kessoonath Surakshit**

All rights reserved worldwide.

This document constitutes a partial fulfillment of the academic requirements for the course, **BCNS2101C Programming Techniques 2**, undertaken with the guidance of **Mr. GOPEE Ajit Kumar**.

Reproduction, storage in a retrieval system, or transmission of this material in any form or by any means—be it electronic, mechanical, photocopying, recording, or otherwise—is strictly prohibited without the express prior written consent of the author.

Table of Contents

GitHub Repo Link:	8
INTRODUCTION	8
1. Object-Oriented Programming (OOP)	9
What is exactly a Project Oriented Programming?	9
2. PROBLEM STATEMENT	9
What problem we can face in a gym as admin?	9
3.AIMS AND OBJECTIVES	10
Aim.....	10
OBJECTIVES	10
4. SCOPE OF THE SYSTEM	10
The system includes:	10
5. OOP CONCEPT USED	11
5.1 Encapsulation.....	11
5.2 Inheritance	11
5.3 Polymorphism.....	12
6. SYSTEM FEATURES	13
7.DEVELOPMENT PROCEDURE.....	13
Development Steps	14
8. UML DIAGRAMS	15
8.1 Use Case Diagram.....	15
What is a Use case Diagram?.....	15
Use case Specification	17
Use case 1 : Add Member	17
Use case 2: View Members	17
Use Case 3: Update Member	18
Use Case 4: Delete Member.....	18
Use Case 5: Add Trainer	19
Use Case 6: Record Attendance	19
Use Case 7: Process Payment.....	20
Use Case 8: Generate Monthly Income Report	20
8.2 Class Diagram	21

What is a class Diagram?	21
8.3 Sequence Diagram	22
What is a sequence Diagram?	22
Use Case Scenario, Process Payment	28
8.4 collaboration Diagram	29
What is a collaboration Diagram?	29
8.5 INHERITANCE DESIGN	30
1.Single Inheritance	30
2.Multilevel Inheritance	31
3.Hierarchical Inheritance	32
4.Multiple Inheritance	33
5.Hybrid Inheritance.....	34
9. ALGORITHM AND FLOWCHARTS	35
9.1 Member Registration Algorithm	35
Flowchart.....	36
9.2 Trainer Management Algorithm.....	36
9.3 Attendance Recording Algorithm	39
9.4 Payment Processing Algorithm	41
9.5 Monthly Income Report Algorithm	43
10. PAYMENT TRANSACTION CALCULATION	45
1. BMI Calculation	45
2. BMI Status	45
3. Total Membership Fee	46
4. Discount Calculation	46
5. Final Amount	47
6. Monthly Income Report	47
7. Automatic Member ID	48
8. Automatic Payment ID.....	48
9. Attendance ID.....	49
11. Debugging/Testing Java Codes.....	50
Project Structure	50
MAIN MENU	52
ADD MEMBER.....	53

VIEW MEMBER + BMI	54
ADD PREMIUM MEMBER	55
ADD TRAINER.....	56
PAYMENT PROCESSING	57
ATTENDANCE.....	58
VIEW PAYMENTS.....	59
MONTHLY REPORT	60
BMI STATISTICS	61
VALIDATION RULES	62
ERROR ENCOUNTERED	63
ERROR 1: FILE WRITING ERROR	63
Error 2: MEMBER ID NOT FOUND DURING PAYMENT	64
ERROR 3: INVALID MENU INPUT.....	65
ERROR 4: Invalid Decimal Input.....	66
ERRORS 5: Negative Age	67
12.TEXT FILE OUTPUT LOCATION.....	68
Members.txt.....	68
Attendance.txt.....	69
Payments.txt	69
PremiumMembers.txt	70
reports.txt	70
Trainers.txt	71
13.Full Java Source Codes	72
Main.java	72
Attendance.java	74
BMIUtil.java	75
FileManager.java	75
GymService.java	80
Member.java	90
MembershipPlan.java	92
Payment.java	93
Person.java	95
PremiumMember.java.....	96

ReportGenerator.java.....	97
Trainer.java.....	99
ValidationUtil.java	99
REFERENCES	101

List Of Tables

Table 1 - Add Member	17
Table 2 - View Member.....	17
Table 3 - Update Member	18
Table 4 - Delete Member	18
Table 5 - Add Trainer	19
Table 6 - Process Payment	20
Table 7 - Generate Monthly Income	20

List Of Figures

Figure 1 - Use Case Diagram Of GymPro Manager	16
Figure 2 - Class Diagram of GymPro Manager	22
Figure 3 - Sequence Diagram for payment processing.	28
Figure 4 - the Collaboration Diagram of GymPro Manager	29
Figure 5 - Single Inheritance	30
Figure 6 - Multilevel Inheritance.....	31
Figure 7 - Hierarchical Inheritance	32
Figure 8 - Hierarchical Inheritance	33
Figure 9 - Hybrid Inheritance	34
Figure 10 - Full project structure of GymPro Manager	51
Figure 11 - Main menu of GymPro Manager	52
Figure 12 - Adding a normal gym member	53
Figure 13 - Viewing member details and BMI calculation.....	54
Figure 14 - Adding a premium member	55
Figure 15 - Adding a trainer	56
Figure 16 - Payment processing and discount calculation	57
Figure 17 - Attendance recording	58
Figure 18 - Viewing payment records	59
Figure 19 - Monthly income report	60
Figure 20 - BMI statistics report	61
Figure 21 - : Validation rules display	62
Figure 22 - File writing error.....	63
Figure 23 - Member not found error	64
Figure 24 - Invalid menu input error.....	65

Figure 25 - Invalid Decimal Input error	66
Figure 26 - Negative Age error	67

Object-Oriented Gym Membership, Trainer, Attendance and Payment Management System.

GitHub Repo Link:

<https://github.com/sahil3608/Java-Assignment.git>

INTRODUCTION

GymPro Manager is a java console-based application developed using Object-Oriented Programming (OOP) Principles. I have designed the system in a way to automate the gym operations such as the member registration, the trainer management, attendance tracking, payment processing and monthly income reporting. The application uses java collections and java file Handling to permanently store records in text files.

In simple terms, the projects demonstrate important OOP concepts including **Encapsulation**, **Inheritance** and **Polymorphism**. The system also performs calculations such as BMI calculation, discount calculation and monthly income calculation.



1. Object-Oriented Programming (OOP)

What is exactly a Project Oriented Programming?

We learnt before in previous session that an OOP is a programming paradigm which is based on **objects** and **classes**. It simply helps us developers to create reusable, secure and organized programs.

The project demonstrates important Object-Oriented Programming concepts such as:

1. Encapsulation
2. Inheritance
3. Polymorphism

So, in this project, **Encapsulation** is used by making variables private and accessing them using methods.

In addition, **Inheritance** is used because **Member and Trainer inherit** from the **Person class**.

In addition, **Polymorphism** is used because different classes override the **displayInfo()** method differently. Abstraction as well demonstrated using the abstract person class.

We have made the application completely in a console-based and does not use **databases or GUI components** which suit the criteria of the assignment.

2. PROBLEM STATEMENT

What problem we can face in a gym as admin?

Many small gyms still manage their members, trainers, attendance, and payments **manually using notebooks** or spreadsheets. So, this often creates several problems such as data loss, wrong calculations, poor attendance tracking, difficulty updating records, slow searching, and poor organization of information.

Therefore, we came across this project thinking about the main objective is to design and develop a **Java-based Gym Management System** that automates **gym operations** and permanently stores records using text files. Eventually, the system aims to improve efficiency, accuracy, and organization as well within gym management processes.

3. AIMS AND OBJECTIVES

Aim

Our aim is to design and develop an advanced Object-Oriented Gym Management System using Java.

OBJECTIVES

- creating a Java console-based application.
- We will demonstrate Object-Oriented Programming concepts.
- demonstrating encapsulation, inheritance and polymorphism.
- We will use Java Collections such as ArrayList.
- storing data permanently using text files.
- reading data back from files.
- Performing CRUD operations: Create, Read, Update and Delete.
- We will process membership payments.
- Also, calculating BMI.
- Calculating discounts.
- Generating monthly income reports.
- Validating user inputs.
- We will test and debug the system professionally.

4. SCOPE OF THE SYSTEM

The system includes:

- Member management
- Trainer management
- Attendance recording
- Payment processing
- BMI calculation
- Discount calculation
- Monthly income reporting
- File saving
- File loading
- File rewriting
- Validation and error handling

5. OOP CONCEPT USED

Now, we will explain some concepts below which will include in our coding phases.

5.1 Encapsulation

Encapsulation; in a brief explanation, we used that to declare class variables as private and accessing them through **getter** and **setter methods**.

For example:

```
Private String name;
```

```
Public String getName() {  
    Return name;
```

```
}
```

Benefits:

1. It protects the data
2. It prevents direct access
3. It improves security

5.2 Inheritance

In our project, we will use inheritance because Member and Trainer inherit from the person class.

For example:

```
public class Member extends Person
```

Benefits we receive:

1. It reduces duplication
2. It improves code reuse
3. It makes the system organized

5.3 Polymorphism

Again here, it is demonstrated through method overriding.

For example:

```
@override  
public void displayInfo()
```

For example:

1. It allows different behaviors.
2. It improves flexibility.
3. Supports reusable code.

6. SYSTEM FEATURES

The following features has been included in our projects which can be shown below:

1. Add Member
2. View Members
3. Update Member
4. Delete Member
5. Search Member by ID
6. Add Trainer
7. View Trainers
8. Process Payment
9. Record Attendance
10. Process Payment
11. View Payments
12. Monthly Income Report
13. Automatic ID Generation
14. BMI Calculation
15. BMI Status
16. Discount Calculation
17. File Saving
18. File Loading
19. File Rewriting
20. Input Validation (Rules)
21. Error Handling
22. Console Menu System

7.DEVELOPMENT PROCEDURE

We have carried out the development of the GymPro Manager step by step using the tool **IntelliJ IDEA 2026.1.1.**

First, we have created the project folders manually. Then all Java classes and text files were created. Furthermore, we have written the Java source

code and compiled using the Java compiler. The application was then executed using the Java console.

We have tested the normal function and error cases. Screenshots were taken during testing. UML diagrams and flowcharts were created using **draw.io**, and finally the complete report was prepared. Everything will be shown in the upcoming pages of the report.

Development Steps

- Create folders in **IntelliJ IDEA 2026.1.1**
- Create all .java files
- Create all .txt files
- Paste the source code
- Compile using `javac *.java`
- Run using `java Main`
- Test normal functions
- Test error cases
- Take screenshots
- Create UML diagrams
- Create flowcharts
- Prepare report
- Add conclusion
- Prepare viva explanation

8. UML DIAGRAMS

8.1 Use Case Diagram

What is a Use case Diagram?

It simply shows the interaction between **the user and the system**. It explains what functions the user (who can be us) we will perform in the application. We are called an actor, and the function are called use cases.

Based on our project GymPro Manager, the main actor is the Admin, that is us; we are the admin doing the project and manage **members, trainers, payments, attendance** and **reports**.

The diagram below shows all the main services available in the system.

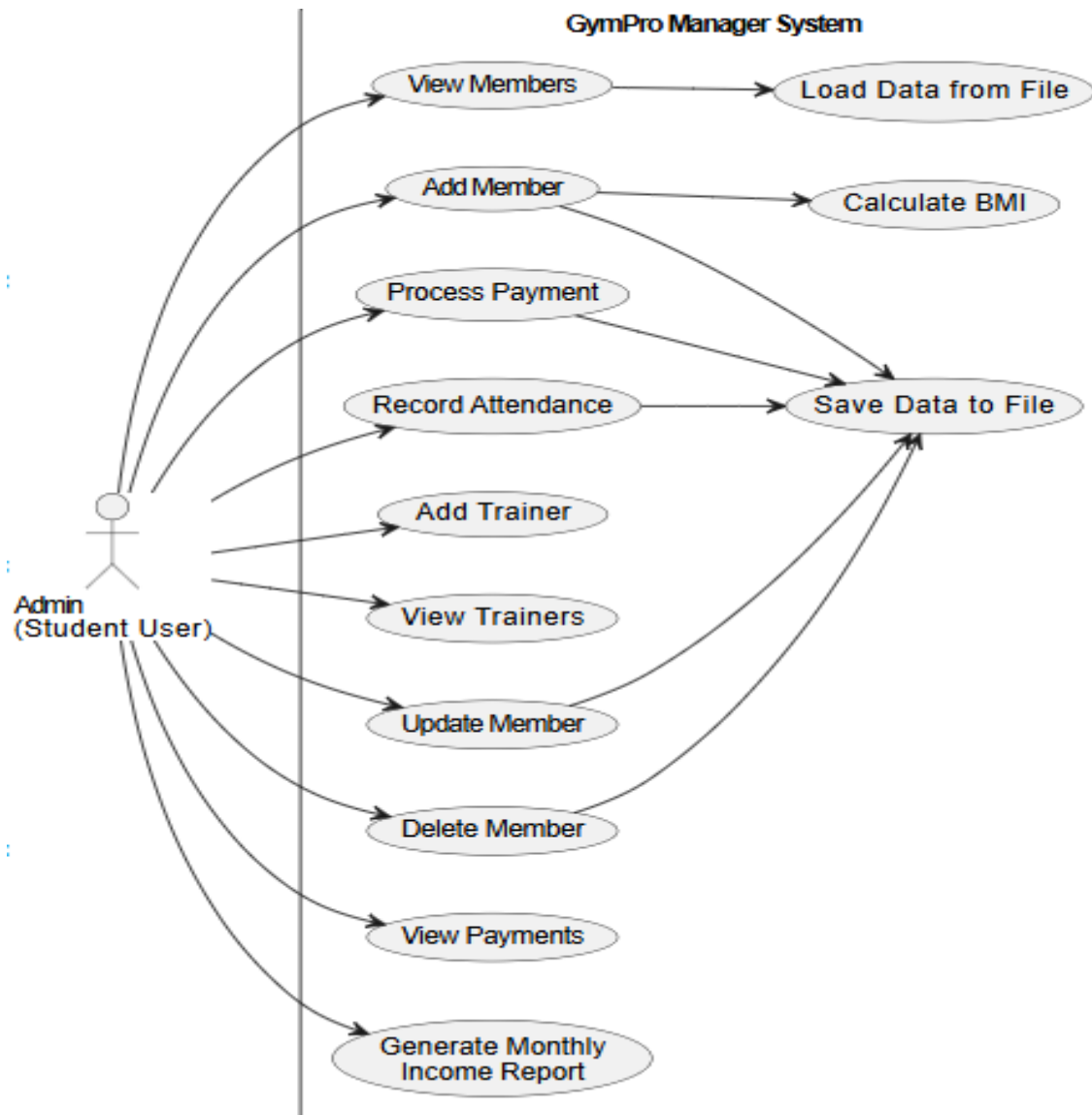


Figure 1 - Use Case Diagram Of GymPro Manager

The use case diagram interacts between Admin and the system. The admin which is us student user; we perform operations such as adding members, managing trainers, processing payments and generating reports.

Use case Specification

Use case 1 : Add Member

Field	Description
Actor	Admin
Description	Admin enters new member details into the system.
Input	ID, name, age, phone, membership type, weight, height
Process	System creates a Member object and stores it in ArrayList<Member>.
Output	Member added successfully.
Related Classes	Main, GymService, Member, FileManager

Table 1 - Add Member

Use case 2: View Members

Field	Description
Actor	Admin
Description	Admin views all registered members.
Input	Menu option
Process	System retrieves and displays member records.
Output	Member details displayed.
Related Classes	GymService, Member

Table 2 - View Member

Use Case 3: Update Member

Field	Description
Actor	Admin
Description	Admin updates member information.
Input	Member ID and updated details
Process	System searches member by ID and updates the record.
Output	Member updated successfully.
Related Classes	GymService, Member, FileManager

Table 3 - Update Member

Use Case 4: Delete Member

Field	Description
Actor	Admin
Description	Admin removes a member from the system.
Input	Member ID
Process	System deletes the member record from the list.
Output	Member deleted successfully.
Related Classes	GymService, Member, FileManager

Table 4 - Delete Member

Use Case 5: Add Trainer

Field	Description
Actor	Admin
Description	Admin adds trainer information.
Input	ID, name, age, phone, specialization, salary
Process	System creates a Trainer object and stores it.
Output	Trainer added successfully.
Related Classes	GymService, Trainer, FileManager

Table 5 - Add Trainer

Use Case 6: Record Attendance

Field	Description
Actor	Admin
Description	Admin records attendance for members.
Input	Member ID and attendance date
Process	System creates an Attendance object.
Output	Attendance recorded successfully.
Related Classes	GymService, Attendance, FileManager

Use Case 7: Process Payment

Field	Description
Actor	Admin
Description	Admin records member payments.
Input	Member ID, amount, payment method
Process	System creates a Payment object and stores payment information.
Output	Payment processed successfully.
Related Classes	GymService, Payment, MembershipPlan, FileManager

Table 6 - Process Payment

Use Case 8: Generate Monthly Income Report

Field	Description
Actor	Admin
Description	Admin generates the monthly income report.
Input	Month
Process	System calculates total monthly income from payments.
Output	Monthly income report displayed.
Related Classes	GymService, Payment

Table 7 - Generate Monthly Income

8.2 Class Diagram

What is a class Diagram?

For the class diagram, it shows the structure of the system. That is; it includes the classes, attributes, methods and relationships between classes. Overall, it shows how the program is organized.

In our project, we implement this concept by dividing the system into different classes such as Member, Trainer, Payment, Attendance, and FileManager. The Member and Trainer classes inherit from the Person class, which therefore show inheritance. Then afterwards, the attributes are private, which shows the encapsulation.

For better understanding, I will provide the class diagram below.

So, below;

Figure 2 shows the Class Diagram of GymPro Manager.

This diagram shows the classes, attributes, methods and relationships used in the system

It also shows the use of inheritance where Member and Trainer inherit from Person.

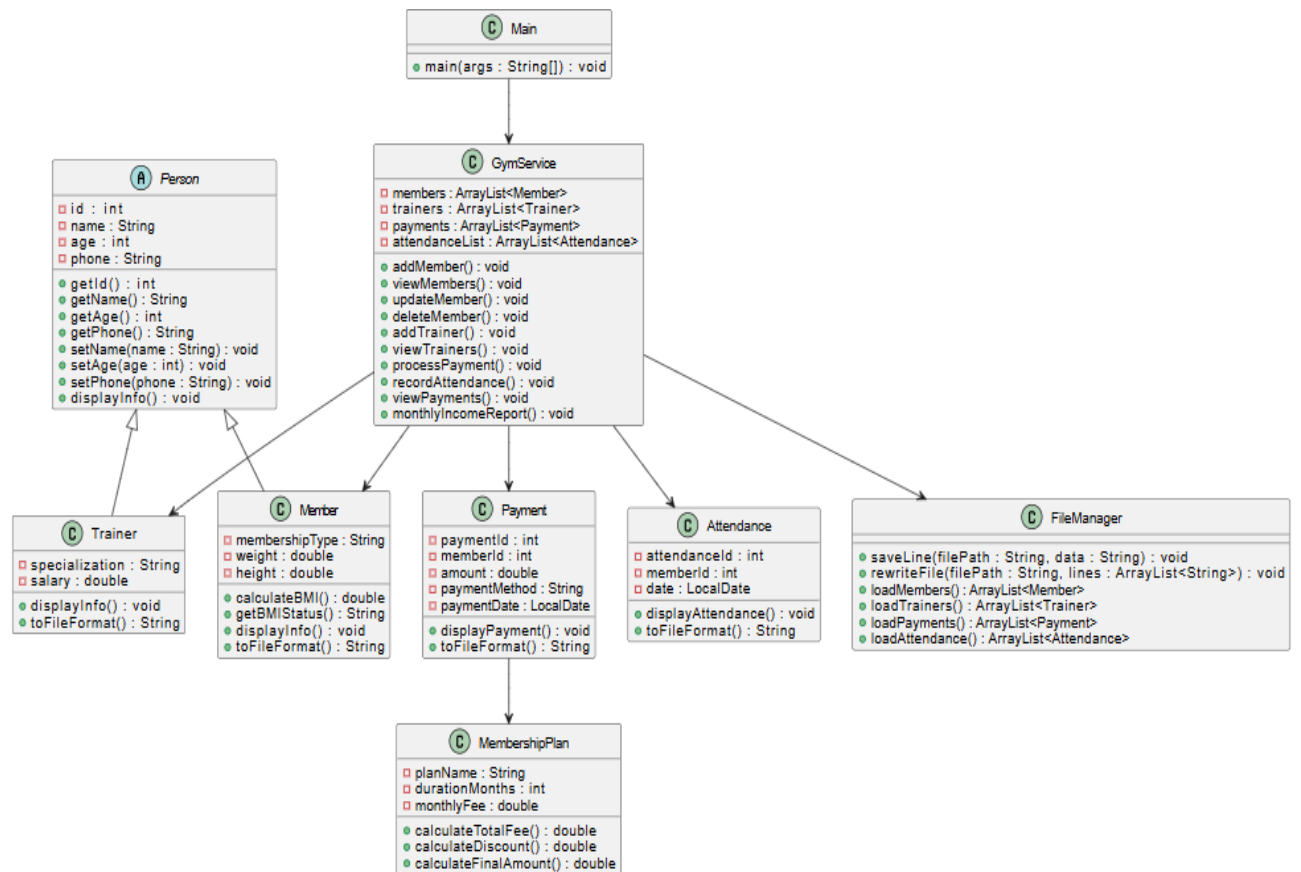


Figure 2 - Class Diagram of GymPro Manager

8.3 Sequence Diagram

What is a sequence Diagram?

It simply shows the order of actions between objects. It explains how objects communicate step by step during a process.

According to our project GymPro Manager, the best sequence diagram is **Payment Processing** because it includes validation, calculation, object creation and file storage.

Note: Person is an Abstract Class

- Main class
main(args : String[]) : void

Here, we'll start the system, displays the menu and calls the GymService methods.

- GymService Class
Attributes
members : ArrayList<Member>
trainers : ArrayList<Trainer>
payments : ArrayList<Payment>
attendanceList : ArrayList<Attendance>

The system stores all members, all trainers, all payments, and all attendance records using java collections (ArrayLists).

Methods

- addMember()
 - viewMembers()
 - updateMember()
 - deleteMember()
- it manages trainers

Trainer Functions

- addTrainer()
 - viewTrainers()
- It manages trainers

Payment Functions

- processPayment()
 - viewPayments()
- It manages the gym payments.

Attendance Functions

recordAttendance()

Report Function

monthlyIncomeReport()

Calculate gym income

- Person Class (Abstract Parent Class)

Attributes

id : int

name : String

age : int

phone : String

These are common information shared by members and Trainer.

Methods

- getId()
- getName()
- setName()
- displayInfo()

These are getters, setters, and display methods.

- Members Class

This is the class inherits from Person. The arrow pointing upwards means the inheritance.

So:

class Member extends Person

Attributes

- membershipType : String
- weight : double
- height : double

Methods

The BMI Calculation

calculateBMI()

formula: BMI = Weight divide by height^2

BMI Status:

It returns the Underweight, Normal, Overweight

- Trainer class

This class also inherits from Person

Attributes

- specialization : String
- salary : double

- Payment class

It stores payment information

Attributes:

- paymentId : int
- memberId : int
- amount : double
- paymentMethod : String
- paymentDate : LocalDate

This mean:

- Which member has paid

- The amount Paid
- Payment Method
- Date

Methods

- displayPayment()
- toFileFormat()

It is used to show and save payment records.

- Attendance Class

It stores attendance records.

Attributes:

- attendanceId : int
- memberId : int
- date : LocalDate

It tracks which member attended and when.

- MembershipPlan Class

It simply handles gym plans

Attributes

- planName : String
- durationMonths : int
- monthlyFee : double

Example;

Plan	Duration	Fee
Basic	1 Month	Rs 1500
Premium	6 Months	Rs 1200

Methods:

- calculateTotalFee()
- calculateDiscount()
- calculateFinalAmount()

Example:

Final Amount = TotalFee -Discount

- FileManager Class

The fileManager is very important for the file handling.

I will use textfile instead of database.

Methods

- saveLine()
- rewriteFile()
- loadMembers()
- loadTrainers()
- loadPayments()
- loadAttendance()

In this class, it saves the records into .txt file and loads records again back into the program.

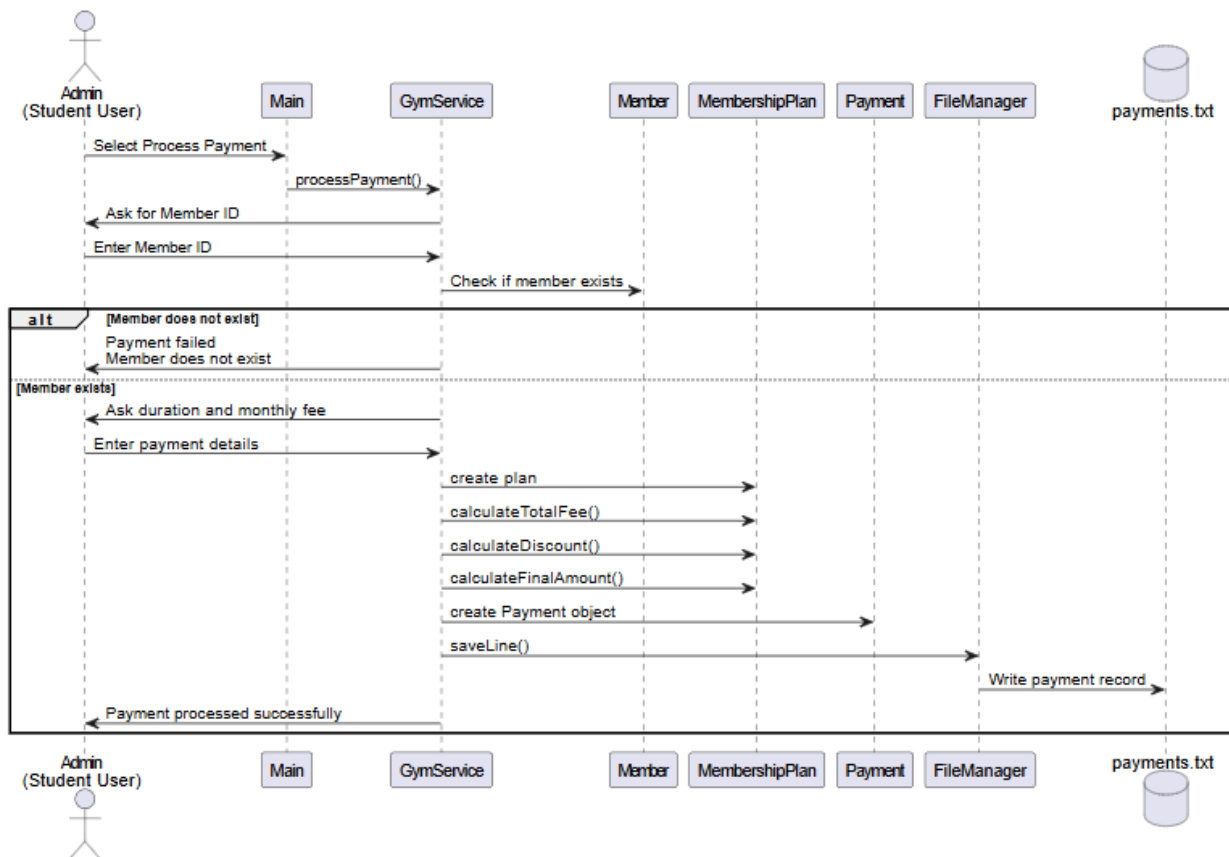


Figure 3 - Sequence Diagram for payment processing.

This diagram explains the order of messages between objects when a payment is made.

Use Case Scenario, Process Payment

The Admin selects Process Payment and enters the Member ID. The system checks whether the member exists. If the member is not found, an error message is displayed and the process ends.

If the member exists, the Admin enters the payment details. The system calculates the total fee, applies any discount, and creates a payment record. The payment information is then saved in payments.txt, and a success message is displayed.

8.4 collaboration Diagram

What is a collaboration Diagram?

It shows how objects work together. It focuses on object communication instead of time order.

So, for the GymPro Manager, different objects work together. The Main class receives the menu choice, GymService controls the logic, and FileManager saves data into text files.

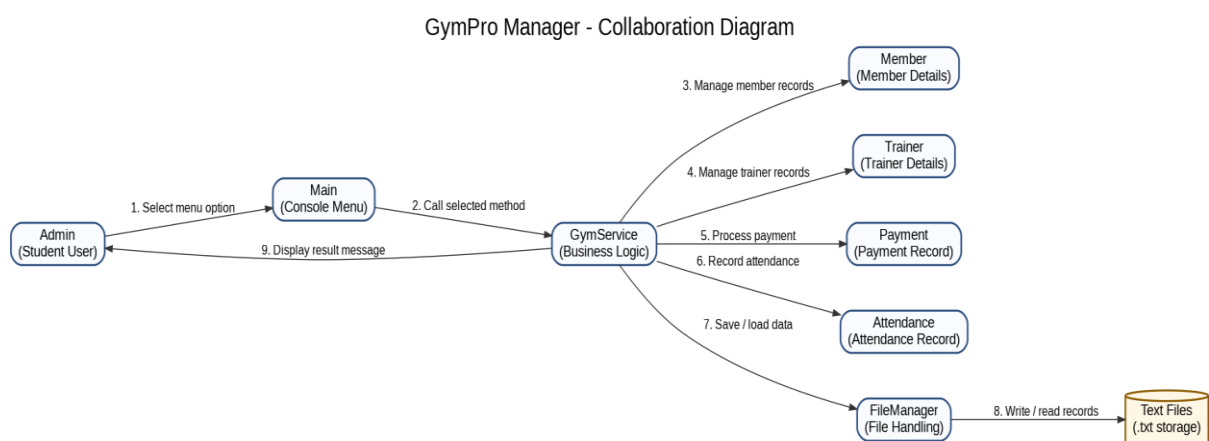


Figure 4 - the Collaboration Diagram of GymPro Manager

This diagram shows how different objects communicate and work together to complete system operations.

8.5 INHERITANCE DESIGN

So, in diagram below, it shows different types of inheritance.

1. Single Inheritance

GymPro Manager - Single Inheritance

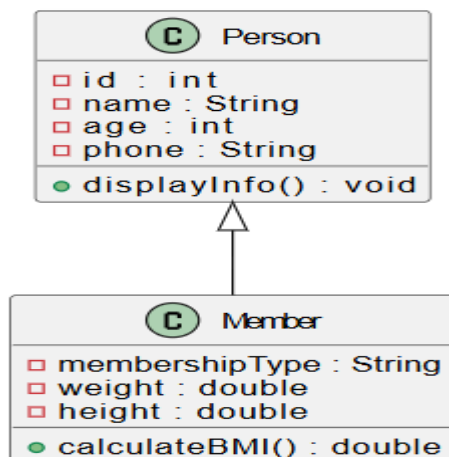


Figure 5- Single Inheritance

In above screenshot, the single inheritance occurs when one child class inherits from one parent class. So, the Member class inherits common attributes and methods from the Person class.

Eventually, this reduces code duplication and improves reusability because shared features are written once in the parent class and reused by the child class.

2.Multilevel Inheritance

GymPro Manager - Multilevel Inheritance

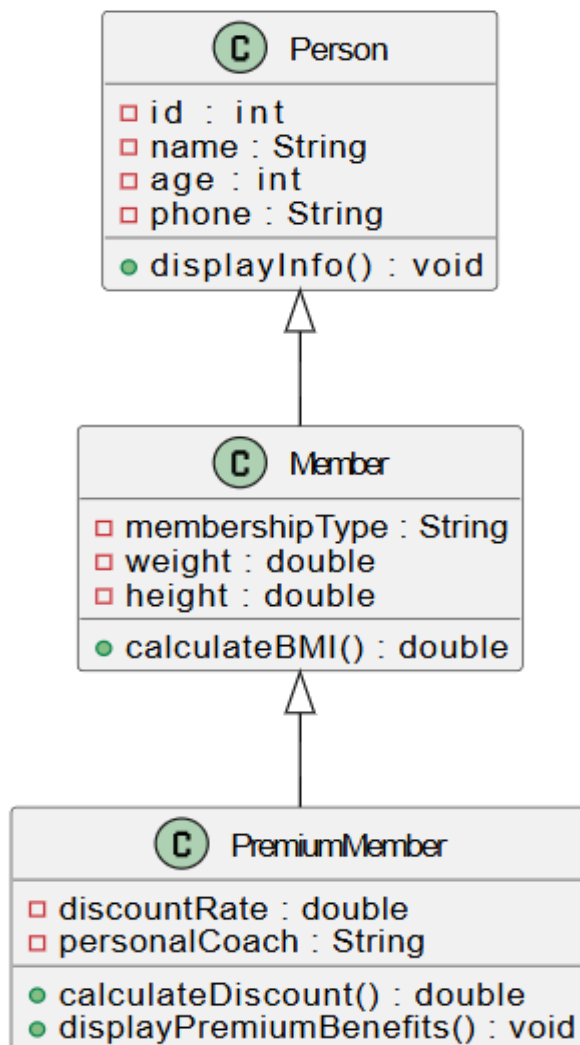


Figure 6 - Multilevel Inheritance

For Multilevel Inheritance, this is when the child class becomes the parent of another class. So, based on the above diagram, PremiumMember inherits from Member, and Member already inherits from Person which therefore creates a three-level inheritance hierarchy.

Example;

Person to Member to PremiumMember

This improves the structure scalability and allows advanced member features such as discounts which I've added and personal coaching to be added without modifying the original classes.

3. Hierarchical Inheritance

GymPro Manager - Hierarchical Inheritance

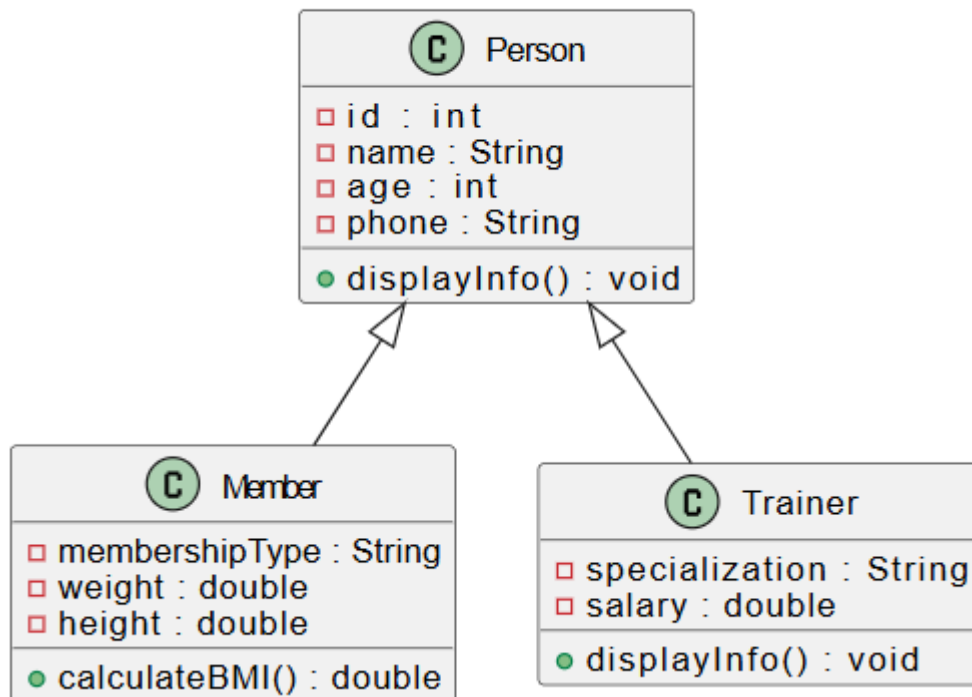


Figure 7- Hierarchical Inheritance

Now, for the hierarchical inheritance, this occurs **when multiple child classes inherit from the same parent class**. In this diagram, both **Member** and **Trainer** inherit from the **Person** class. The child classes share common information from the parent class while also having their own specific attributes and methods. At the end, it improves the code organization and reusability.

4. Multiple Inheritance

GymPro Manager - Multiple Inheritance Using Interfaces

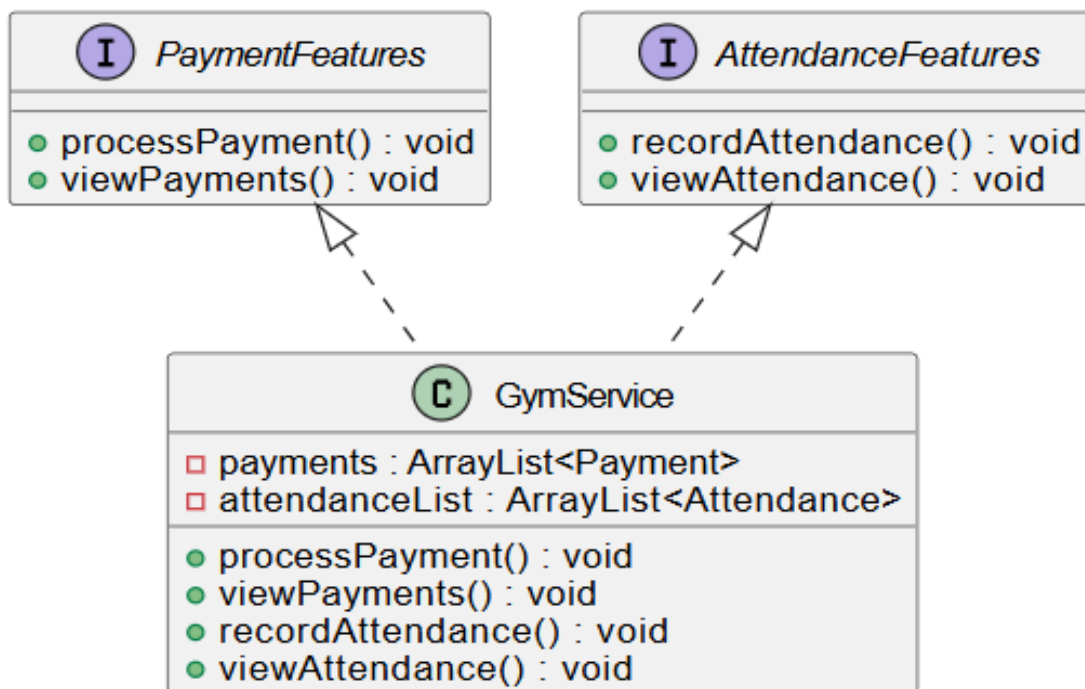


Figure 8 - Hierarchical Inheritance

For the above screenshot, we know that Java does not support multiple inheritance classes directly, but it does support it through interfaces. In this diagram, the **GymService class** implements both **PaymentFeatures** and **AttendanceFeatures** interfaces which allows the class to inherit behaviors from multiple sources while avoiding problems found in traditional multiple inheritance

5. Hybrid Inheritance

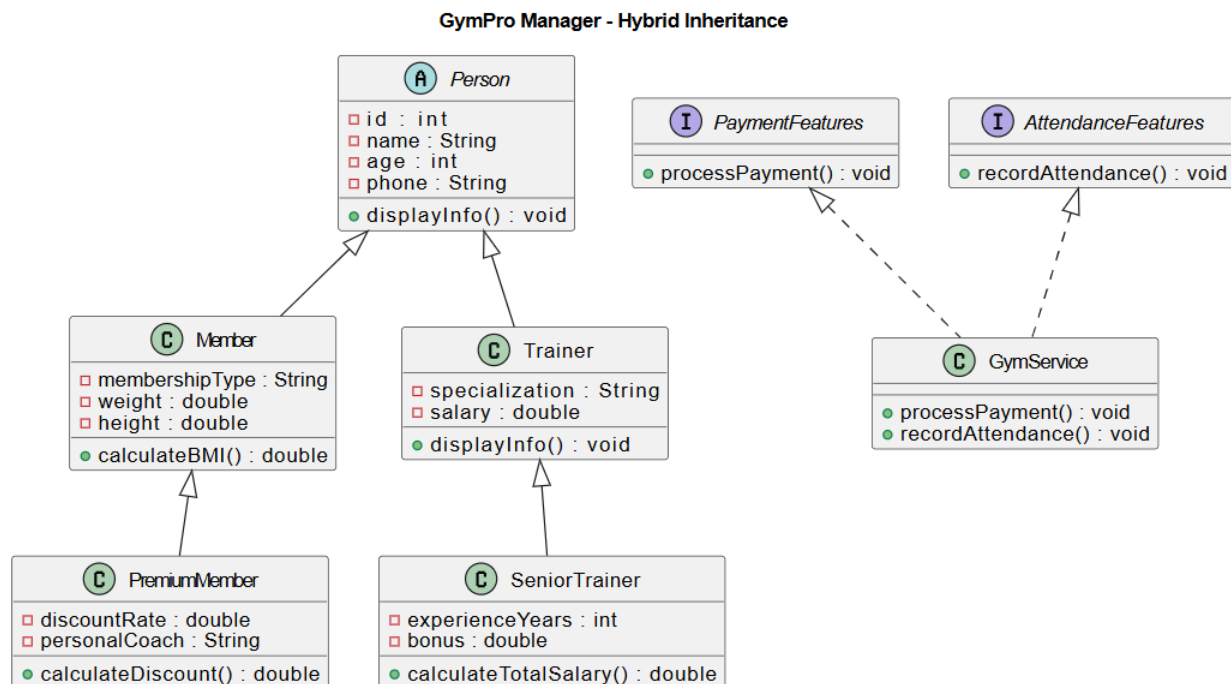


Figure 9 - Hybrid Inheritance

Now, In the above diagram, it combines two or more inheritance types that is; there are a combination of hierarchical inheritance, multilevel inheritance and interface-based multiple inheritance within the same system.

The class **Member** and **Trainer** inherit from **Person**

While, **PremiumMember** and **SeniorTrainer** extend the inheritance hierarchy further.

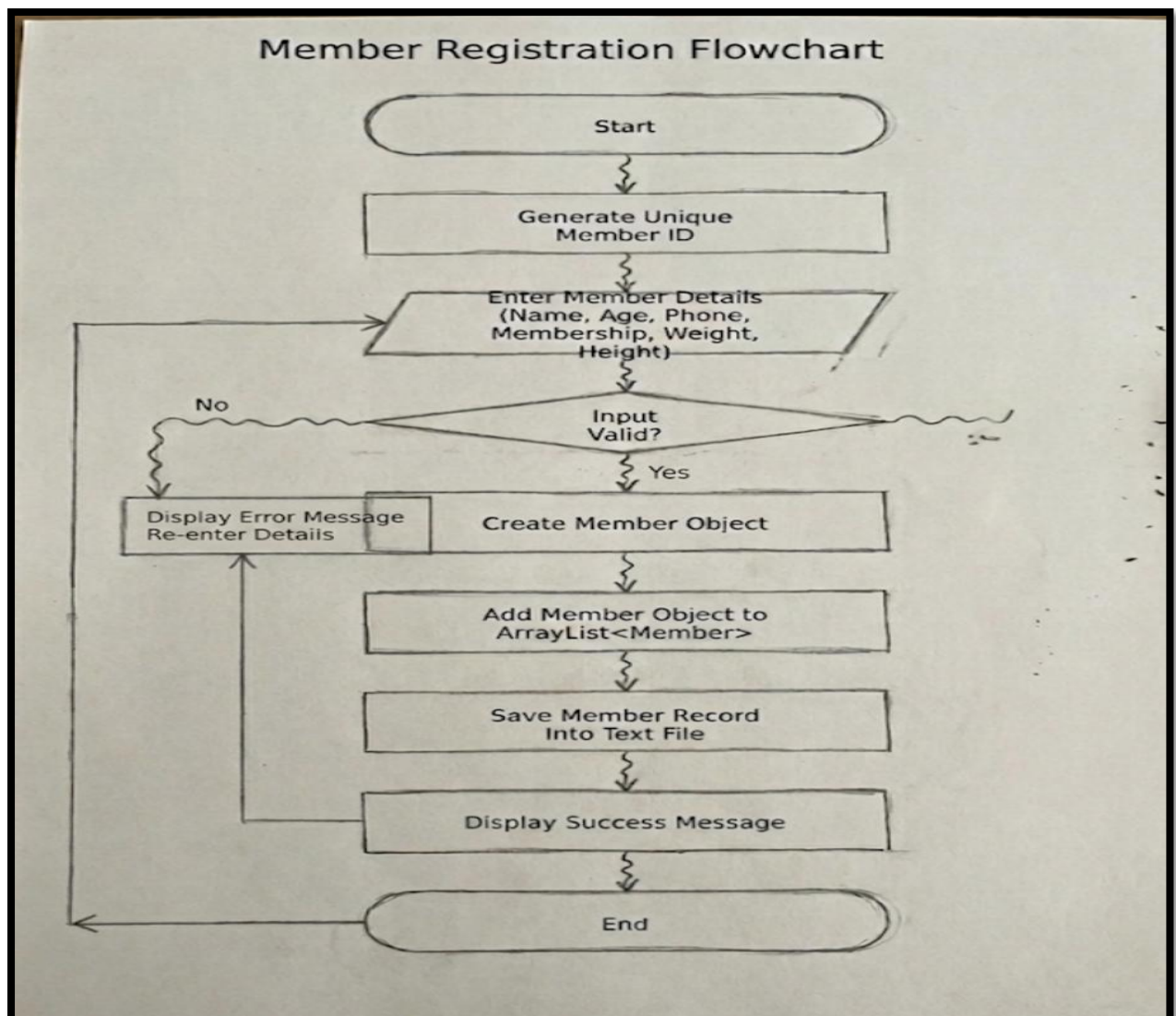
At the same time, **GymService** implements multiple interfaces which creates a flexible and scalable Object-Oriented design for our project GymPro System.

9. ALGORITHM AND FLOWCHARTS

9.1 Member Registration Algorithm

1. Start
2. Generate a unique Member ID
3. Enter member details such as name, age, phone number, membership type, weight and height
4. Validate the entered details
5. If the input is invalid, display an error message and ask the Admin to re-enter the details
6. Create a Member object
7. Add the Member object to the ArrayList<Member>
8. Save the member record permanently into the text file
9. Display a success message
10. End

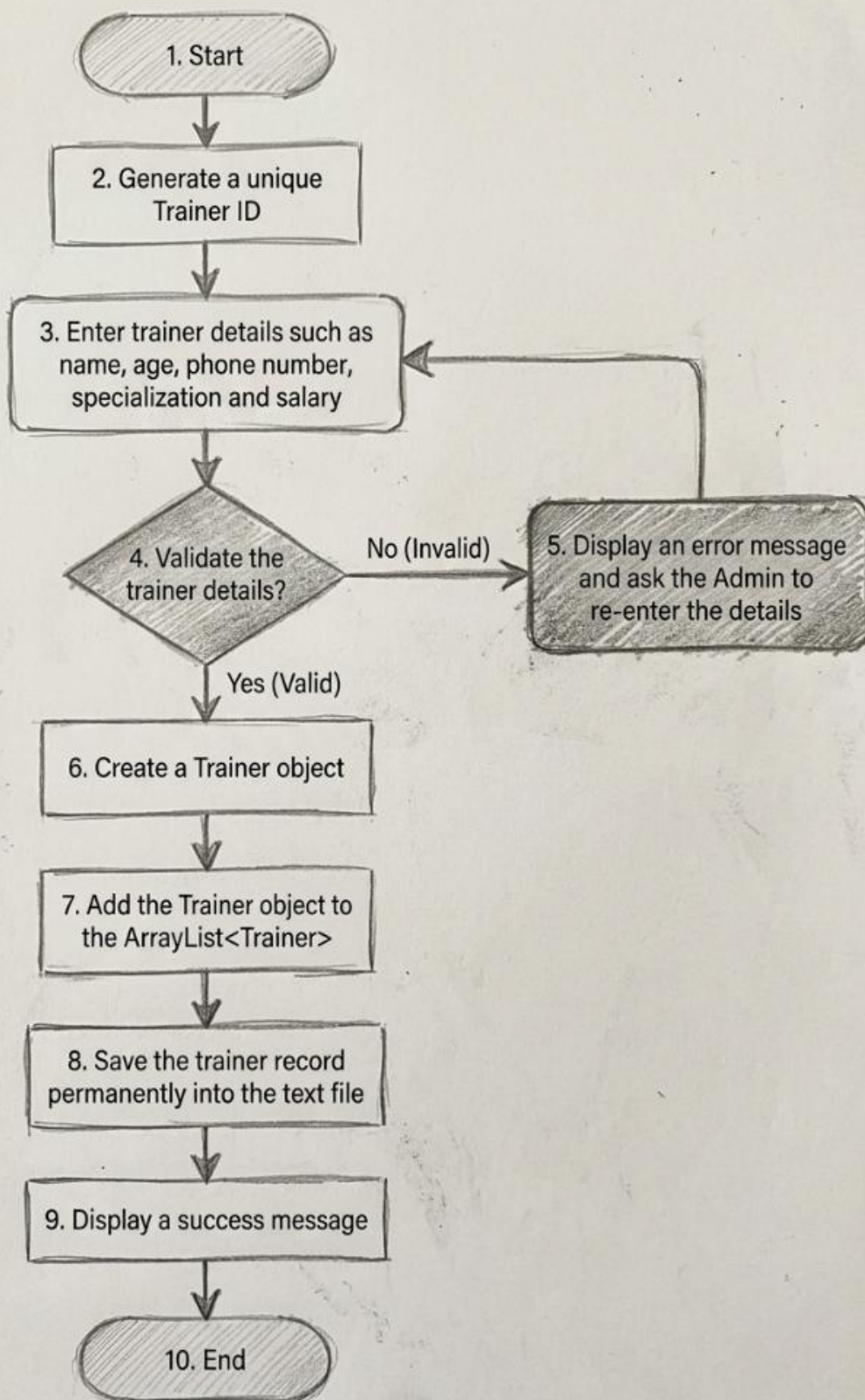
Flowchart



9.2 Trainer Management Algorithm

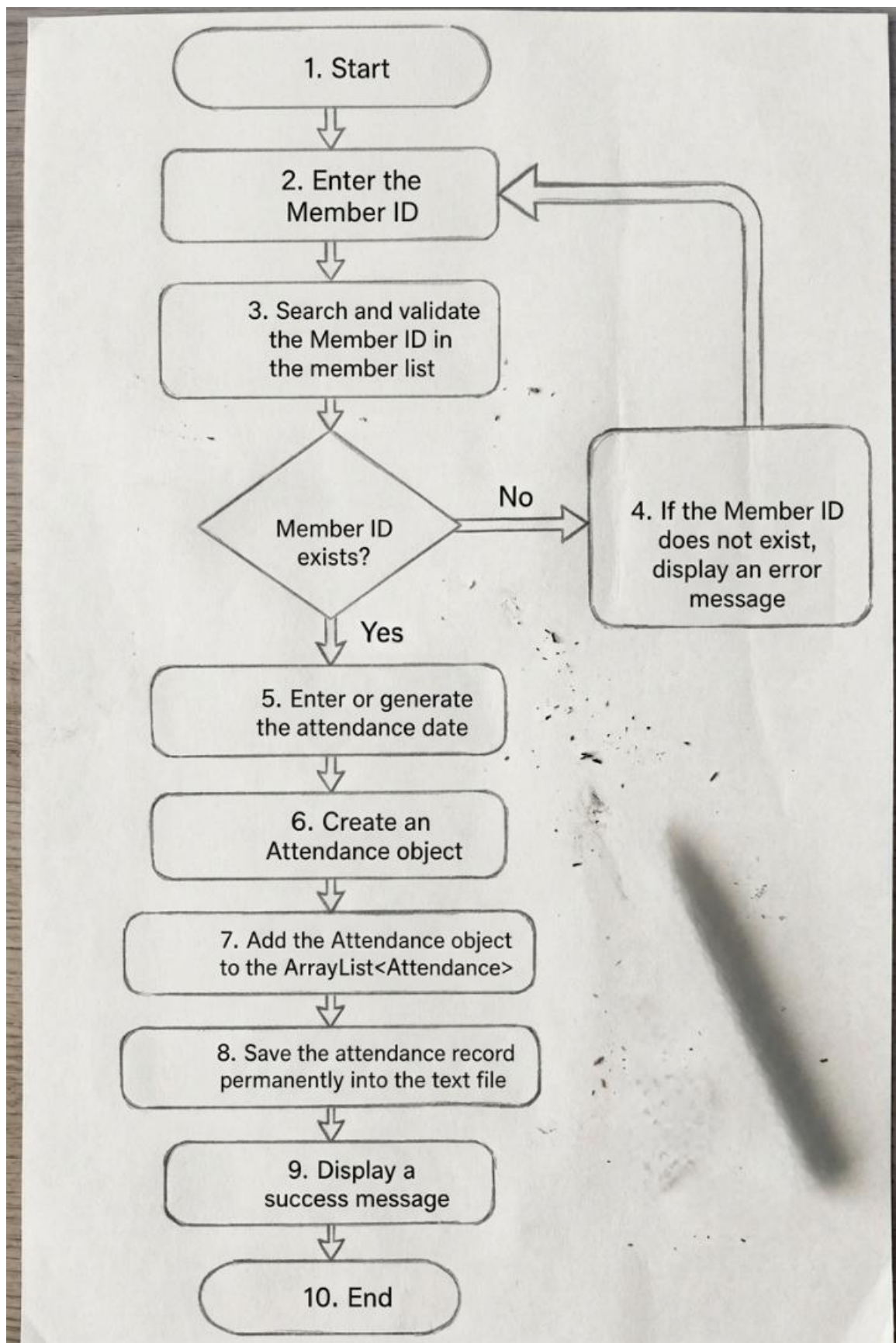
- 1.Start
- 2.Generate a unique Trainer ID
- 3.Enter trainer details such as name, age, phone number, specialization and salary
- 4.Validate the trainer details
- 5.f the input is invalid, display an error message and ask the Admin to re-enter the details

6. Create a Trainer object
7. Add the Trainer object to the ArrayList
8. Save the trainer record permanently into the text file
9. Display a success message
10. End



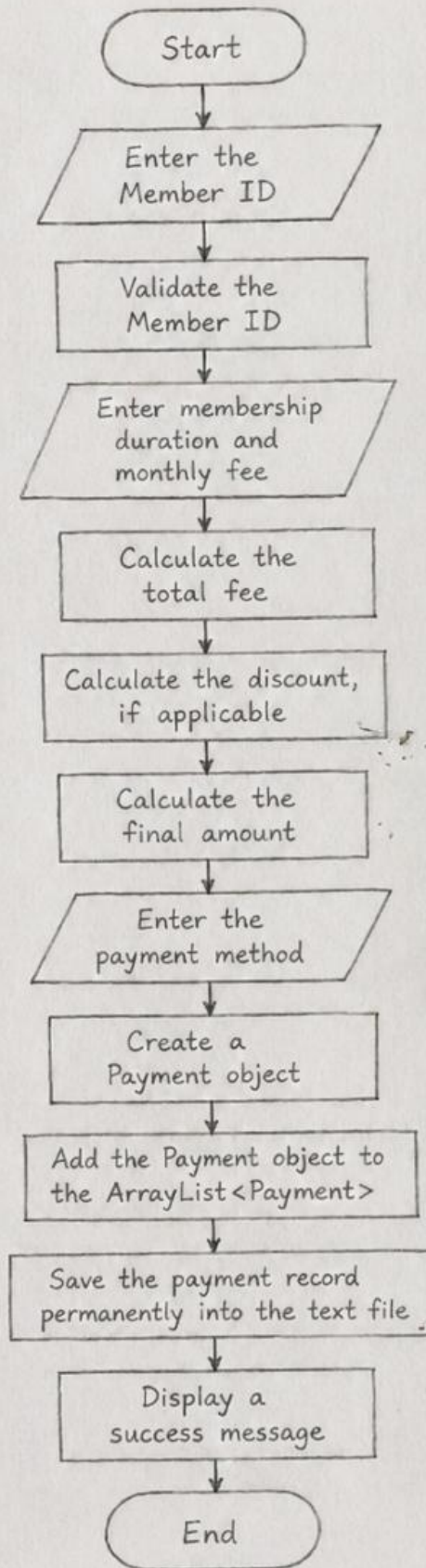
9.3 Attendance Recording Algorithm

1. Start
2. Enter the Member ID
3. Search and validate the Member ID in the member list
4. If the Member ID does not exist, display an error message
5. Enter or generate the attendance date
6. Create an Attendance object
7. Add the Attendance object to the ArrayList
8. Save the attendance record permanently into the text file
9. Display a success message
10. End



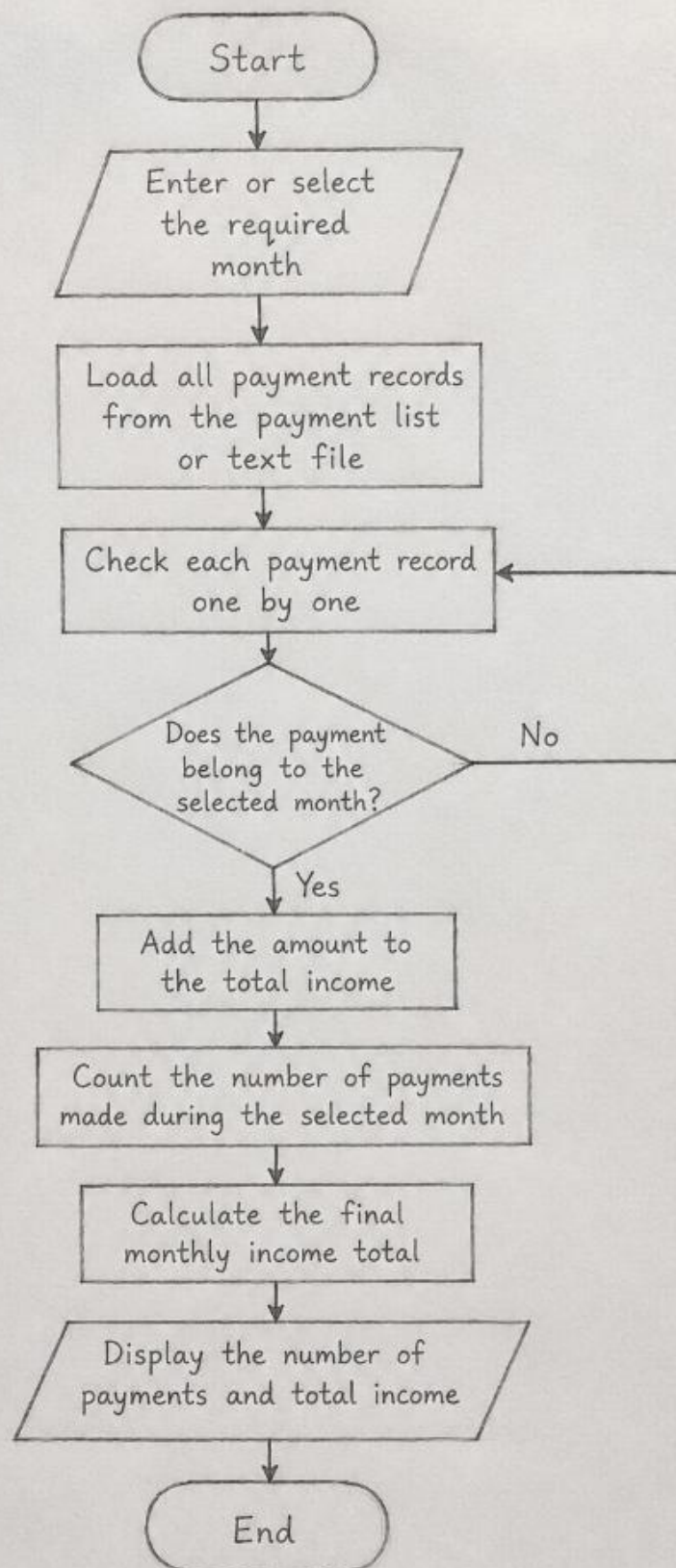
9.4 Payment Processing Algorithm

1. Start
2. Enter the Member ID
3. Validate the Member ID
4. Enter membership duration and monthly fee
5. Calculate the total fee
6. Calculate the discount, if applicable
7. Calculate the final amount
8. Enter the payment method
9. Create a Payment object
10. Add the Payment object to the ArrayList<Payment>
11. Save the payment record permanently into the text file
12. Display a success message
13. End



9.5 Monthly Income Report Algorithm

1. Start
2. Enter or select the required month
3. Load all payment records from the payment list or text file
4. Check each payment record one by one
5. If the payment belongs to the selected month, add the amount to the total income
6. Count the number of payments made during the selected month
7. Calculate the final monthly income total
8. Display the number of payments and total income
9. End



10. PAYMENT TRANSACTION CALCULATION

1. BMI Calculation

What is a BMI calculation?

It means Body Mass Index. It is used to check whether a member has a healthy body weight compared to their height.

Formula:

$$\text{BMI} = \text{Weight}/(\text{height} \times \text{height})$$

Example:

Weight = 70kg

Height = 1.75 M

Step-by-step:

$$1.75 \times 1.75 = 3.0625$$

$$70 / 3.0625 = 22.86$$

So, the member's BMI is 22.86.

2. BMI Status

Now, after calculating the BMI, the system will check which category the member belongs to:

There are rules that we have put:

1. Below 18.5 = Underweight
2. 18.5 to 24.9 = Normal
3. 25 to 29.9 = Overweight
4. 30 and above = Obese

In above BMI, we got 22.86. Since 22.86 is in the range of 18.5 to 24.9, the member is classified as Normal.

3. Total Membership Fee

It calculates the full membership price before discount.

Formula used:

$\text{Total Fee} = \text{Monthly Fee} \times \text{Number of Months}$

Example:

Monthly fee = Rs 1000 Months = 12

Calculations:

$1000 \times 12 = \text{Rs } 12000$

Therefore, the total membership fee is Rs 12000

4. Discount Calculation

For the discount calculation, the system gives discount depending on the membership duration.

Rules:

12 months or more = 20% discount

6 months or more = 10% discount

Below 6 months = No discount

Example:

Monthly fee = Rs 1000 Months = 12

Since the member chooses 12 months, the discount is 20%

Calculations:

$$12000 \times 20 / 100 = \text{Rs } 2400$$

So, the discount amount is Rs 2400.

5. Final Amount

It is the calculation of the real amount the member must pay after discount.

Formula:

final amount = Total Fee – Discount

Example:

Total Fee = Rs 12000

Discount = Rs 2400

Calculations:

$12000 - 2400 = \text{Rs } 9600$; so, the amount to pay is Rs 9600

6. Monthly Income Report

It does the calculation about how much money the gym earned in a selected month.

What exactly the system does?

It checks all payments record. If the payment belongs to the selected month and year, the payment amount is added to the monthly income total.

Example:

June 2026 payments:

Rs 9600 + Rs 4500

Step-by-step:

$9600 + 4500 = \text{Rs } 14100$

So, the total income for June 2026 is Rs 14100.

7. Automatic Member ID

For the specific ID, the system automatically generates a new member ID to avoid duplicate IDs.

How it works?

New Member ID = Highest Existing Member ID + 1

Example:

Highest Member ID = 1003

Calculations:

$1003 + 1 = 1004$

So, the new member ID is 1004.

8. Automatic Payment ID

Here, the system also generates a unique payment ID automatically.

The logic behind:

New Payment ID = Highest Existing Payment ID + 1

Example:

Highest Payment ID = 249

Calculations:

$249 + 1 = 250$

So, the new payment ID is 3006.

9. Attendance ID

The attendance ID is generated automatically to identify each attendance record clearly.

Example:

Previous attendance count = 4

Logic behind:

The system creates the next attendance ID.

Calculations:

If attendance IDs start from 4001, then:

4001, 4002, 4003, 4004

The next ID will be:

4005

So, the new attendance ID is 4005.

So, overall, in the above calculation, it calculates to demonstrates processing data.

NOTE:

12 months or more = 20% discount

6 months or more = 10% discount

Below 6 months = No discount

11. Debugging/Testing Java Codes

Project Structure

The Java application was developed using IntelliJ IDEA. The project structure is organized into well-defined folders and subfolders, including the src directory containing the source code and the data directory used for storing application data. The figure below illustrates the overall project hierarchy and directory organization.

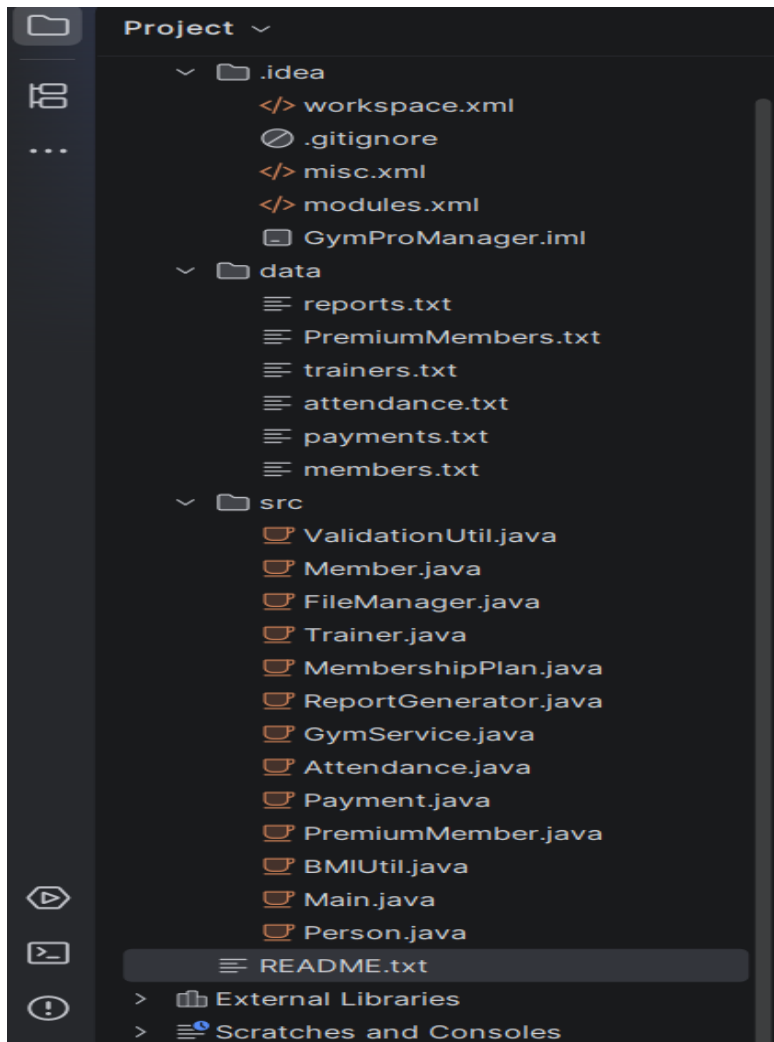


Figure 10- Full project structure of GymPro Manager

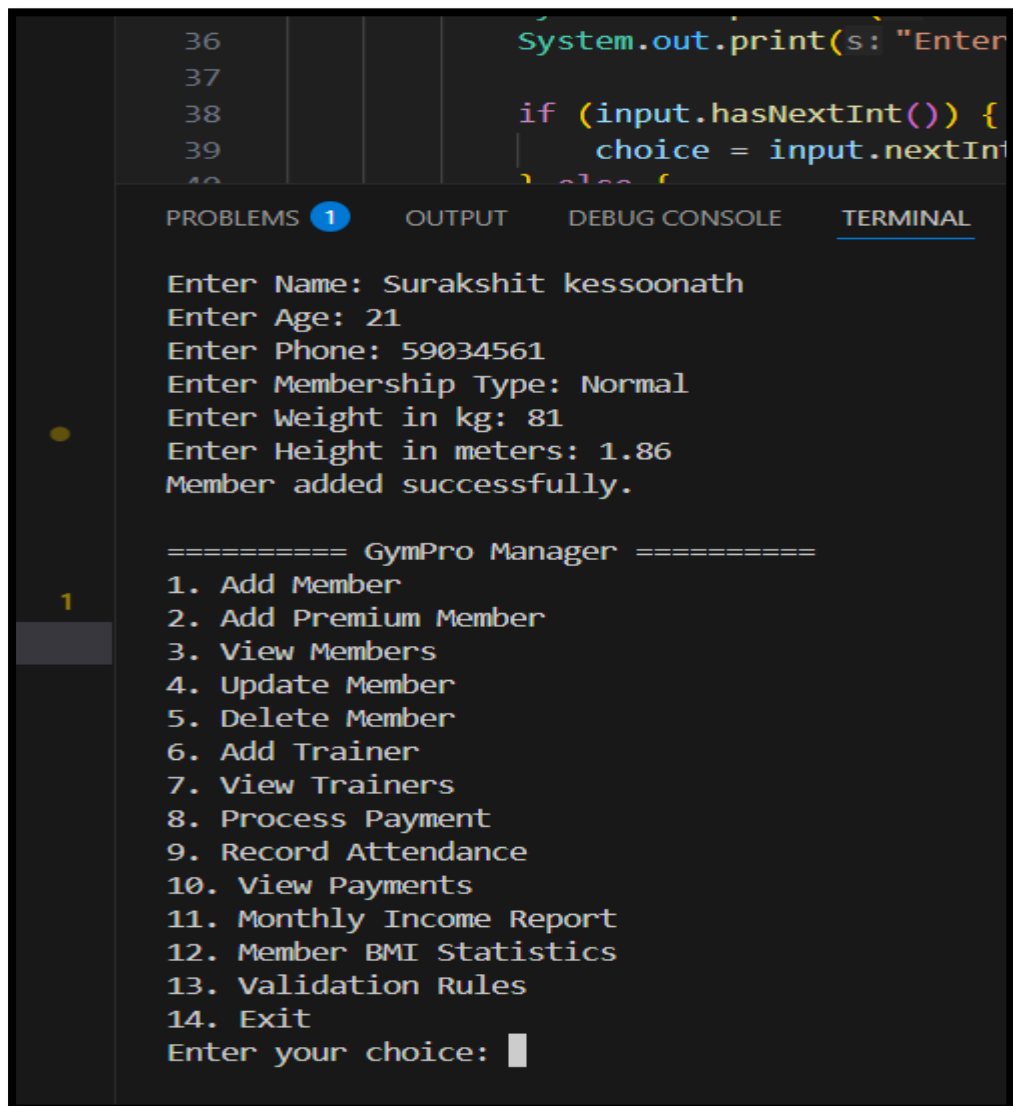
MAIN MENU

```
PS C:\Users\DELL\Downloads\GymProManager> & 'C:\Pr  
' '-cp' 'C:\Users\DELL\AppData\Roaming\Code\User\wo  
er_b1413728\bin' 'Main'  
No previous member data found.  
No previous trainer data found.  
No previous payment data found.  
No previous attendance data found.  
  
===== GymPro Manager =====  
1. Add Member  
2. Add Premium Member  
3. View Members  
4. Update Member  
5. Delete Member  
6. Add Trainer  
7. View Trainers  
8. Process Payment  
9. Record Attendance  
10. View Payments  
11. Monthly Income Report  
12. Member BMI Statistics  
13. Validation Rules  
14. Exit
```

Figure 11 - Main menu of GymPro Manager

ADD MEMBER

Below, as can be seen in the terminal, I choose 1 which is **add Member**, it showed the list of the person information who is my name itself, so that I can be a member in the gym.



```
36         System.out.print(s: "Enter
37
38         if (input.hasNextInt()) {
39             choice = input.nextInt()
40         } else {
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL

```
Enter Name: Surakshit kessoonath
Enter Age: 21
Enter Phone: 59034561
Enter Membership Type: Normal
Enter Weight in kg: 81
Enter Height in meters: 1.86
Member added successfully.

===== GymPro Manager =====
1. Add Member
2. Add Premium Member
3. View Members
4. Update Member
5. Delete Member
6. Add Trainer
7. View Trainers
8. Process Payment
9. Record Attendance
10. View Payments
11. Monthly Income Report
12. Member BMI Statistics
13. Validation Rules
14. Exit
Enter your choice: █
```

Figure 12- Adding a normal gym member

VIEW MEMBER + BMI

Now, in the terminal below, I choose 3 which is view Members. It shows the BMI for the new member john which is 22.86 and the BMI status is normal.

```
===== GymPro Manager =====
1. Add Member
2. Add Premium Member
3. View Members
4. Update Member
5. Delete Member
6. Add Trainer
7. View Trainers
8. Process Payment
9. Record Attendance
10. View Payments
11. Monthly Income Report
12. Member BMI Statistics
13. Validation Rules
14. Exit
Enter your choice: 3
-----
Member ID: 1001
Name: Surakshit kessoonath
Age: 21
Phone: 59034561
Membership Type: Normal
Weight: 81.0 kg
Height: 1.86 m
BMI: 23.41
BMI Status: Normal
```

Figure 13- Viewing member details and BMI calculation

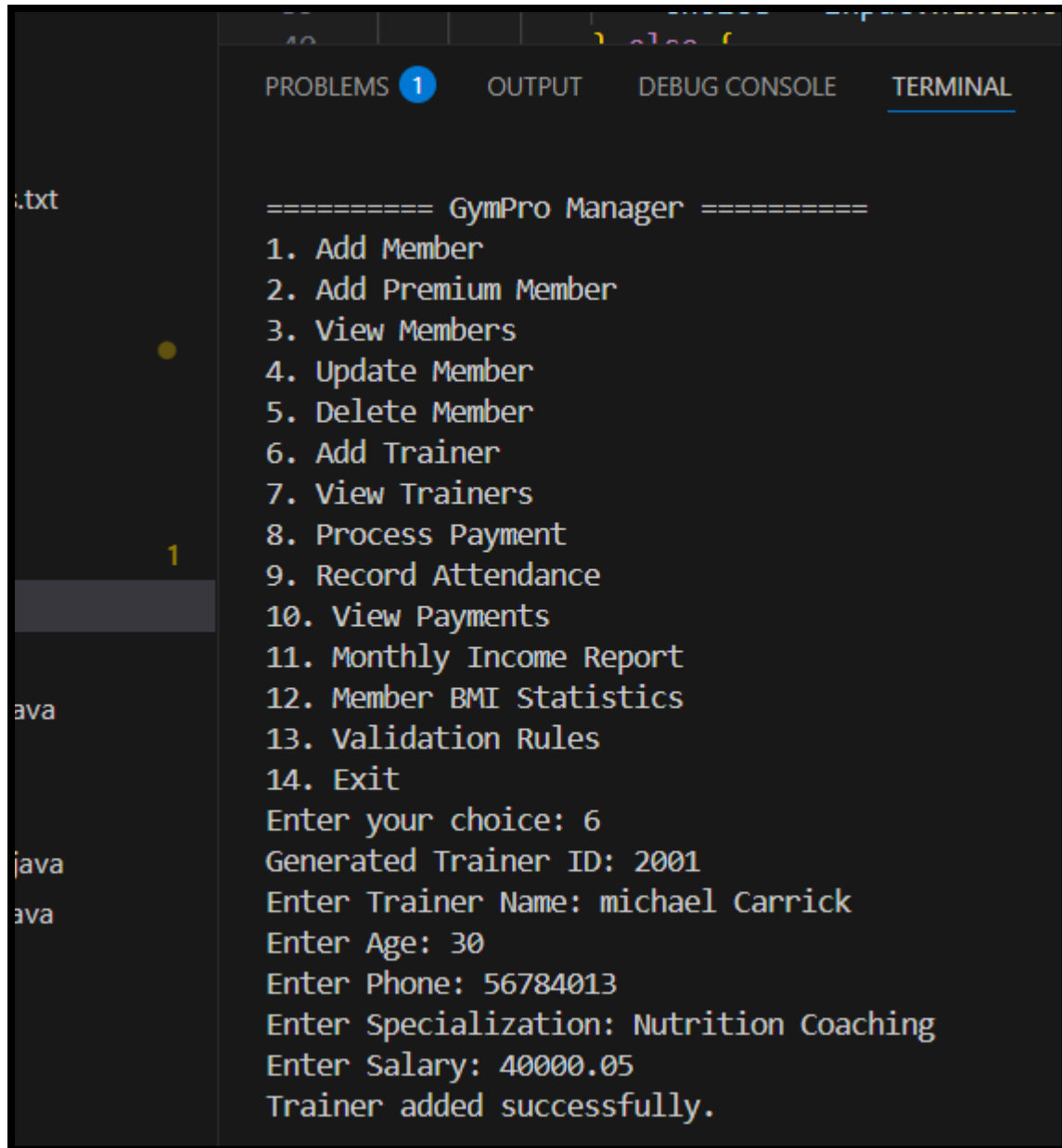
ADD PREMIUM MEMBER

```
===== GymPro Manager =====
1. Add Member
2. Add Premium Member
3. View Members
4. Update Member
5. Delete Member
6. Add Trainer
7. View Trainers
8. Process Payment
9. Record Attendance
10. View Payments
11. Monthly Income Report
12. Member BMI Statistics
13. Validation Rules
14. Exit
Enter your choice: 2
Generated Premium Member ID: 1002
Enter Name: Cristiano Ronaldo
Enter Age: 40
Enter Phone: 57894653
Enter Weight in kg: 78
Enter Height in meters: 1.86
Enter Premium Benefit: Free Personal Trainer
Premium member added successfully.
```

Figure 14 - Adding a premium member

ADD TRAINER

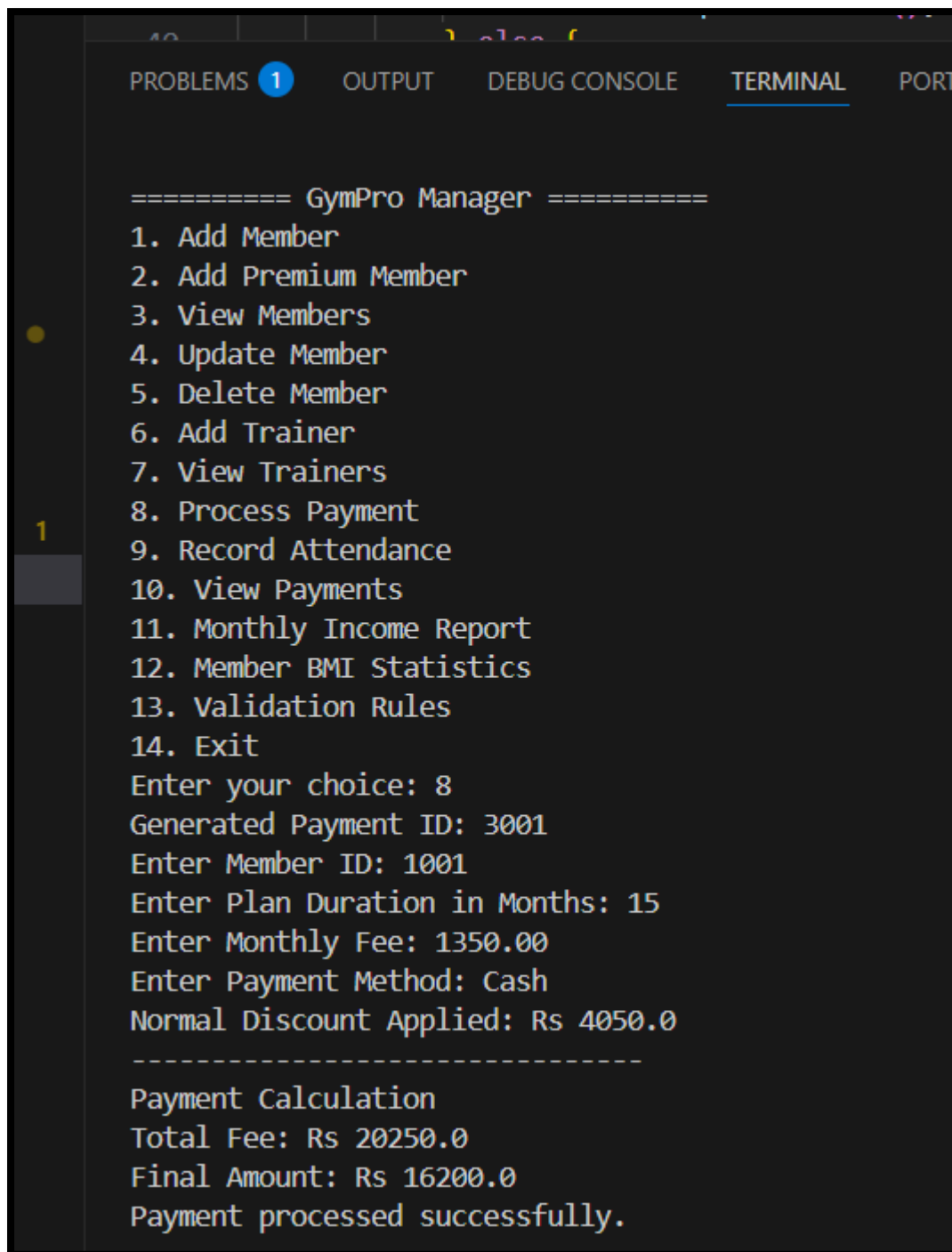
Below, show the name of the trainer and we confirmed that the trainer has been added successfully.



```
===== GymPro Manager =====
1. Add Member
2. Add Premium Member
3. View Members
4. Update Member
5. Delete Member
6. Add Trainer
7. View Trainers
8. Process Payment
9. Record Attendance
10. View Payments
11. Monthly Income Report
12. Member BMI Statistics
13. Validation Rules
14. Exit
Enter your choice: 6
Generated Trainer ID: 2001
Enter Trainer Name: michael Carrick
Enter Age: 30
Enter Phone: 56784013
Enter Specialization: Nutrition Coaching
Enter Salary: 40000.05
Trainer added successfully.
```

Figure 15- Adding a trainer

PAYMENT PROCESSING



```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORT

===== GymPro Manager =====
1. Add Member
2. Add Premium Member
3. View Members
4. Update Member
5. Delete Member
6. Add Trainer
7. View Trainers
8. Process Payment
9. Record Attendance
10. View Payments
11. Monthly Income Report
12. Member BMI Statistics
13. Validation Rules
14. Exit
Enter your choice: 8
Generated Payment ID: 3001
Enter Member ID: 1001
Enter Plan Duration in Months: 15
Enter Monthly Fee: 1350.00
Enter Payment Method: Cash
Normal Discount Applied: Rs 4050.0
-----
Payment Calculation
Total Fee: Rs 20250.0
Final Amount: Rs 16200.0
Payment processed successfully.
```

Figure 16- Payment processing and discount calculation

ATTENDANCE

The attendance is recorded every time, I entered the gym.

```
===== GymPro Manager =====  
1. Add Member  
2. Add Premium Member  
3. View Members  
4. Update Member  
5. Delete Member  
6. Add Trainer  
7. View Trainers  
8. Process Payment  
9. Record Attendance  
10. View Payments  
11. Monthly Income Report  
12. Member BMI Statistics  
13. Validation Rules  
14. Exit  
Enter your choice: 9  
Generated Attendance ID: 4001  
Enter Member ID: 1001  
Attendance recorded successfully.
```

Figure 17 - Attendance recording

VIEW PAYMENTS

This shows all my payment record since day one I joined the gym.

```
===== GymPro Manager =====
1. Add Member
2. Add Premium Member
3. View Members
4. Update Member
5. Delete Member
6. Add Trainer
7. View Trainers
8. Process Payment
9. Record Attendance
10. View Payments
11. Monthly Income Report
12. Member BMI Statistics
13. Validation Rules
14. Exit
Enter your choice: 10
-----
Payment ID: 3001
Member ID: 1001
Amount Paid: Rs 16200.0
Payment Method: Cash
Payment Date: 2026-06-02
```

Figure 18 - Viewing payment records

MONTHLY REPORT

Below, the payment for June 2026 has already been done. To confirm I entered the monthly income report to get the confirmation amount of Rs 16200

```
===== GymPro Manager =====
1. Add Member
2. Add Premium Member
3. View Members
4. Update Member
5. Delete Member
6. Add Trainer
7. View Trainers
8. Process Payment
9. Record Attendance
10. View Payments
11. Monthly Income Report
12. Member BMI Statistics
13. Validation Rules
14. Exit
Enter your choice: 11
Enter month number, example 1 for January: 6
Enter year, example 2026: 2026
-----
MONTHLY INCOME REPORT
Month: 6
Year: 2026
Number of Payments: 1
Total Income: Rs 16200.0
```

Figure 19- Monthly income report

BMI STATISTICS

```
===== GymPro Manager =====
1. Add Member
2. Add Premium Member
3. View Members
4. Update Member
5. Delete Member
6. Add Trainer
7. View Trainers
8. Process Payment
9. Record Attendance
10. View Payments
11. Monthly Income Report
12. Member BMI Statistics
13. Validation Rules
14. Exit
Enter your choice: 12
-----
MEMBER BMI STATISTICS
Total Members: 2
Underweight Members: 0
Normal Members: 2
Overweight Members: 0
Obese Members: 0
```

Figure 20 - BMI statistics report

VALIDATION RULES

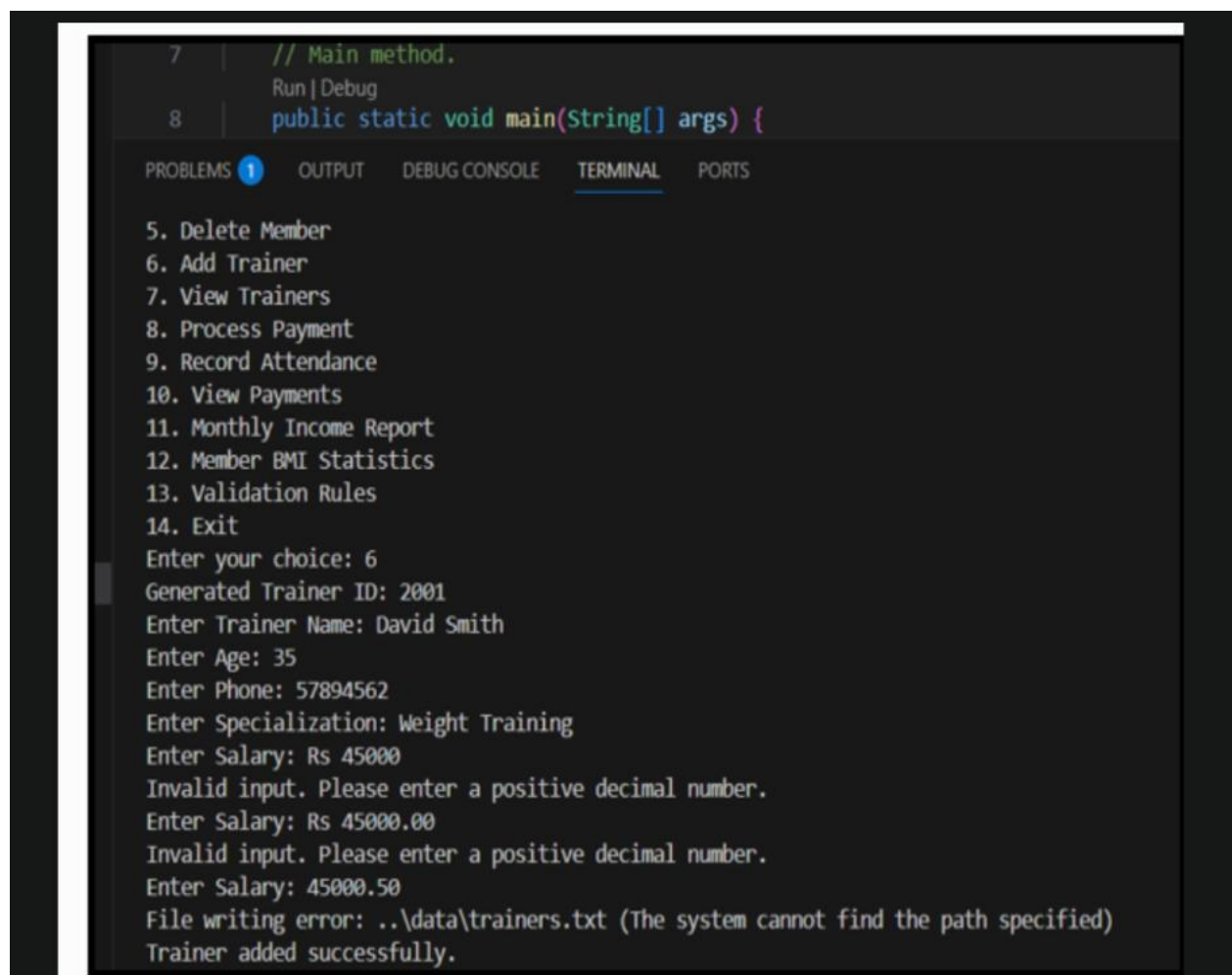
```
===== GymPro Manager =====
1. Add Member
2. Add Premium Member
3. View Members
4. Update Member
5. Delete Member
6. Add Trainer
7. View Trainers
8. Process Payment
9. Record Attendance
10. View Payments
11. Monthly Income Report
12. Member BMI Statistics
13. Validation Rules
14. Exit
Enter your choice: 13
-----
Validation Rules
Name cannot be empty.
Age must be between 12 and 100.
Phone must contain 7 to 10 digits.
Height must be between 0.5m and 2.5m.
Weight must be between 20kg and 250kg.
```

Figure 21 - : Validation rules display

ERROR ENCOUNTERED

During the development of the application, we encountered several errors while writing and testing the code. To document the debugging process, screenshots of these errors were captured. The following sections present each error individually, together with an explanation of the coding mistakes identified and the corrective actions taken to resolve them.

ERROR 1: FILE WRITING ERROR



```
7 // Main method.  
Run | Debug  
8 public static void main(String[] args) {  
  
    PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS  
  
    5. Delete Member  
    6. Add Trainer  
    7. View Trainers  
    8. Process Payment  
    9. Record Attendance  
    10. View Payments  
    11. Monthly Income Report  
    12. Member BMI Statistics  
    13. Validation Rules  
    14. Exit  
    Enter your choice: 6  
    Generated Trainer ID: 2001  
    Enter Trainer Name: David Smith  
    Enter Age: 35  
    Enter Phone: 57894562  
    Enter Specialization: Weight Training  
    Enter Salary: Rs 45000  
    Invalid input. Please enter a positive decimal number.  
    Enter Salary: Rs 45000.00  
    Invalid input. Please enter a positive decimal number.  
    Enter Salary: 45000.50  
    File writing error: ..\data\trainers.txt (The system cannot find the path specified)  
    Trainer added successfully.
```

Figure 22 - File writing error

File writing error: ../data/members.txt

The system cannot find the path specified.

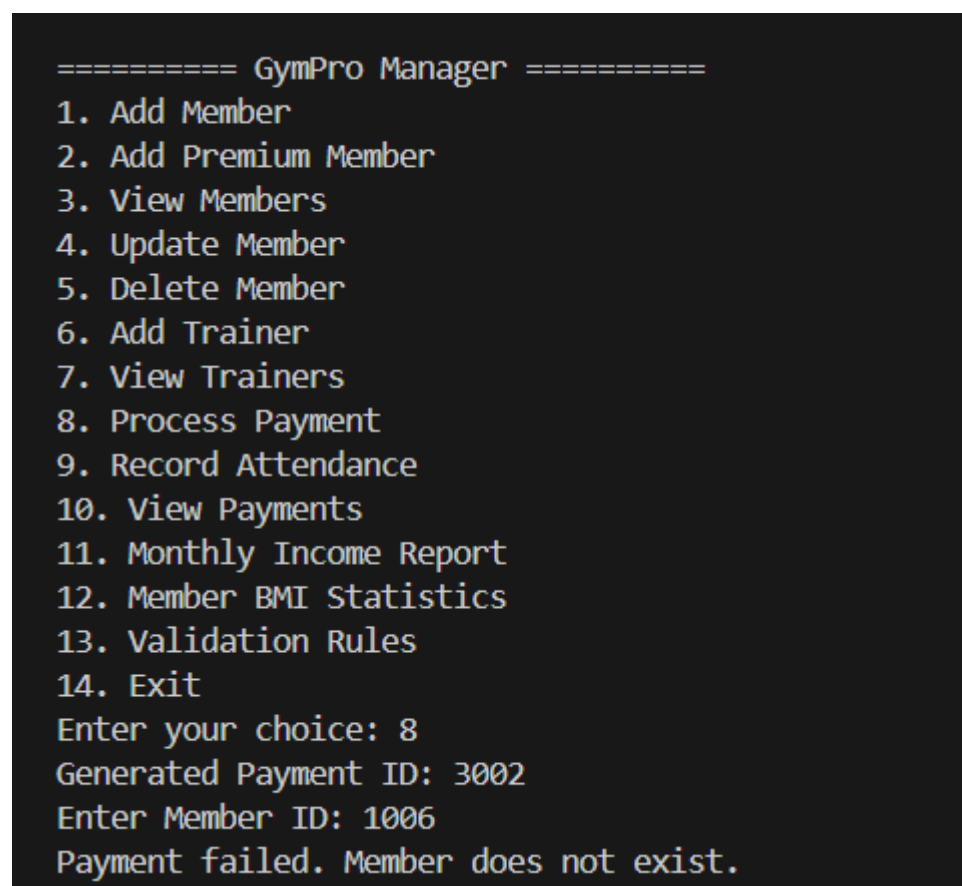
During the development, the system could not write member record into the text file because the relative file path was not being located correctly by java

Correction: Then I changed the file path to an absolute path Which allowed the system to save the records permanently.

```
public static final String MEMBER_FILE =
```

```
"C:/Users/DELL/Downloads/GymProManager/data/members.txt";
```

Error 2: MEMBER ID NOT FOUND DURING PAYMENT

A screenshot of a Java application window titled "GymPro Manager". The window has a dark background with light-colored text. It displays a menu of 14 options: 1. Add Member, 2. Add Premium Member, 3. View Members, 4. Update Member, 5. Delete Member, 6. Add Trainer, 7. View Trainers, 8. Process Payment, 9. Record Attendance, 10. View Payments, 11. Monthly Income Report, 12. Member BMI Statistics, 13. Validation Rules, and 14. Exit. Option 8, "Process Payment", is selected. Below the menu, the text "Enter your choice: 8" is displayed. Then, "Generated Payment ID: 3002" is shown. Next, "Enter Member ID: 1006" is displayed. Finally, the message "Payment failed. Member does not exist." is shown at the bottom of the window.

```
===== GymPro Manager =====
1. Add Member
2. Add Premium Member
3. View Members
4. Update Member
5. Delete Member
6. Add Trainer
7. View Trainers
8. Process Payment
9. Record Attendance
10. View Payments
11. Monthly Income Report
12. Member BMI Statistics
13. Validation Rules
14. Exit
Enter your choice: 8
Generated Payment ID: 3002
Enter Member ID: 1006
Payment failed. Member does not exist.
```

Figure 23- Member not found error

This happened because the member record was not saved correctly into members.txt due to the file path issue. Therefore, when payment was processed using ID 1006, the system could not find the member.

Correction: I corrected the file path, then the member was added again and the record appeared in members.txt

This error showed that payment processing depends on member records. The system correctly prevented payment from being processed for a member that was not found.

ERROR 3: INVALID MENU INPUT

```
Enter your choice: c
Invalid choice. Please enter a number.

===== GymPro Manager =====
1. Add Member
2. Add Premium Member
3. View Members
4. Update Member
5. Delete Member
6. Add Trainer
7. View Trainers
8. Process Payment
9. Record Attendance
10. View Payments
11. Monthly Income Report
12. Member BMI Statistics
13. Validation Rules
14. Exit
Enter your choice: Invalid choice. Please enter a number.
```

Figure 24 - Invalid menu input error

By mistake, I entered letters instead of a menu number.

In the system, it uses:

`input.hasNextInt()`; to check whether the menu input is numeric.

This validation prevents the program from crashing when the user enters text instead of a number.

ERROR 4: Invalid Decimal Input

```
===== GymPro Manager =====
1. Add Member
2. Add Premium Member
3. View Members
4. Update Member
5. Delete Member
6. Add Trainer
7. View Trainers
8. Process Payment
9. Record Attendance
10. View Payments
11. Monthly Income Report
12. Member BMI Statistics
13. Validation Rules
14. Exit
Enter your choice: 1
Generated Member ID: 1003
Enter Name: vicky kessoo
Enter Age: 25
Enter Phone: 59876757
Enter Membership Type: Normal
Enter Weight in kg: 70kg
Invalid input. Please enter a positive decimal number.
Enter Weight in kg: 
```

Figure 25 - Invalid Decimal Input error

Here, above errors occurs because java expects decimal numbers using a dot, not a comma or text.

Correction: The system `hasNextDouble()` and repeats the input request until a valid decimal number is entered.

ERRORS 5: Negative Age

```
===== GymPro Manager =====
1. Add Member
2. Add Premium Member
3. View Members
4. Update Member
5. Delete Member
6. Add Trainer
7. View Trainers
8. Process Payment
9. Record Attendance
10. View Payments
11. Monthly Income Report
12. Member BMI Statistics
13. Validation Rules
14. Exit
Enter your choice: 1
Generated Member ID: 1002
Enter Name: vicky members
Enter Age: -21
Invalid input. Please enter a positive number.
Enter Age: 
```

Figure 26- Negative Age error

I accidentally pressed negative in my keyboard.

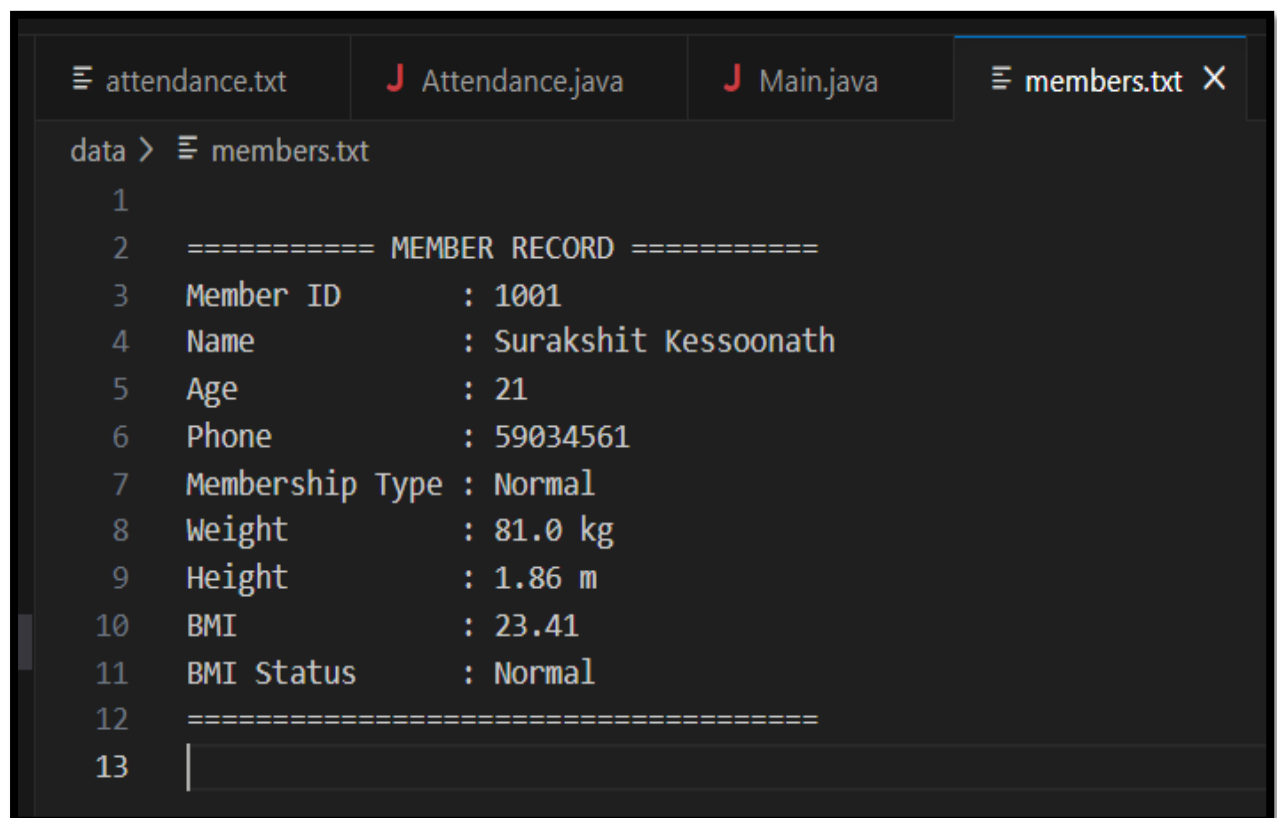
So. Having a negative age is unrealistic and should not be stored.

Correction: age >= 12 && age <= 100

Age validation was added to ensure that only realistic gym member ages are accepted.

12.TEXT FILE OUTPUT LOCATION

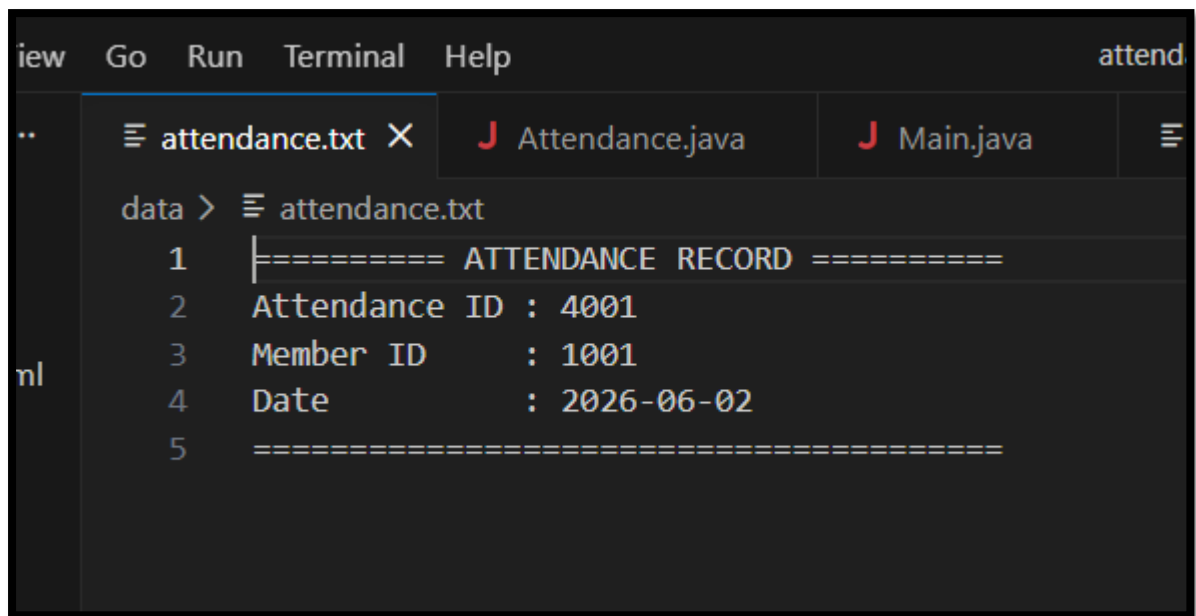
Members.txt



The screenshot shows a Java IDE with four tabs: 'attendance.txt', 'Attendance.java', 'Main.java', and 'members.txt'. The 'members.txt' tab is active, showing the following output:

```
data > members.txt
1
2  ===== MEMBER RECORD =====
3  Member ID      : 1001
4  Name           : Surakshit Kessoonath
5  Age            : 21
6  Phone          : 59034561
7  Membership Type : Normal
8  Weight         : 81.0 kg
9  Height         : 1.86 m
10 BMI           : 23.41
11 BMI Status     : Normal
12  =====
13
```

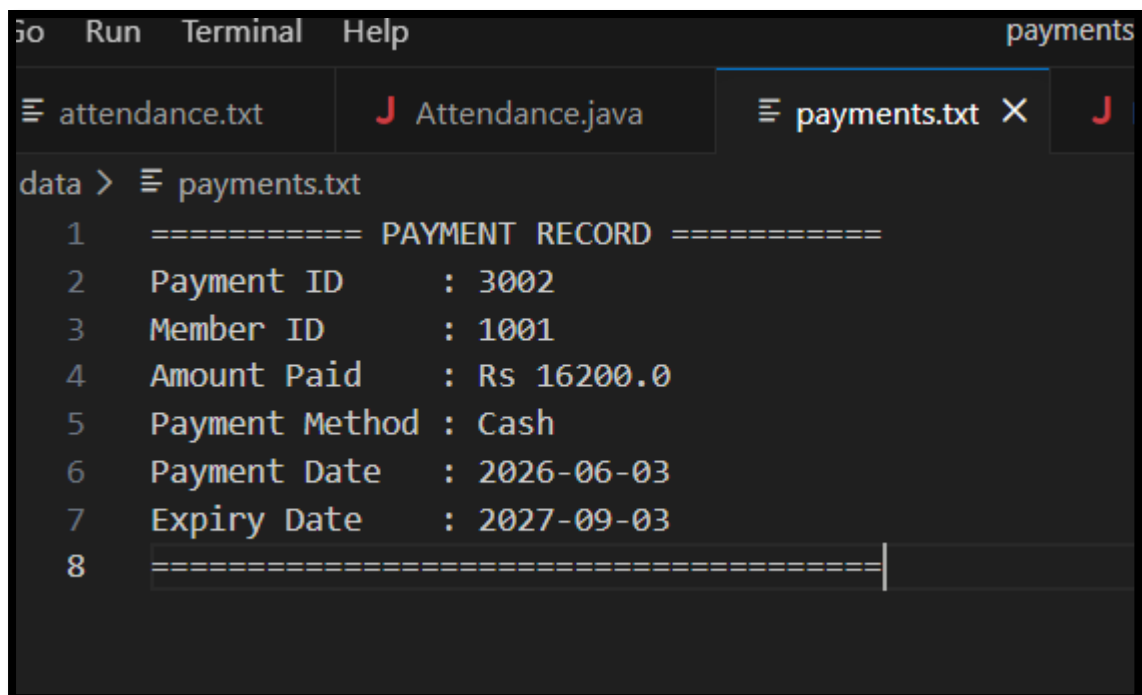
Attendance.txt



The screenshot shows an IDE window with the title bar 'attend'. The menu bar includes 'iew', 'Go', 'Run', 'Terminal', and 'Help'. The tab bar shows 'attendance.txt' (selected), 'Attendance.java', and 'Main.java'. The editor area displays the content of 'attendance.txt' with line numbers 1 through 5. The text is as follows:

```
data > attendance.txt
1  |===== ATTENDANCE RECORD =====
2  Attendance ID : 4001
3  Member ID    : 1001
4  Date         : 2026-06-02
5  =====
```

Payments.txt



The screenshot shows an IDE window with the title bar 'payments'. The menu bar includes 'Go', 'Run', 'Terminal', and 'Help'. The tab bar shows 'attendance.txt', 'Attendance.java', 'payments.txt' (selected), and a red 'J' icon. The editor area displays the content of 'payments.txt' with line numbers 1 through 8. The text is as follows:

```
data > payments.txt
1  ===== PAYMENT RECORD =====
2  Payment ID    : 3002
3  Member ID     : 1001
4  Amount Paid   : Rs 16200.0
5  Payment Method : Cash
6  Payment Date  : 2026-06-03
7  Expiry Date   : 2027-09-03
8  =====
```

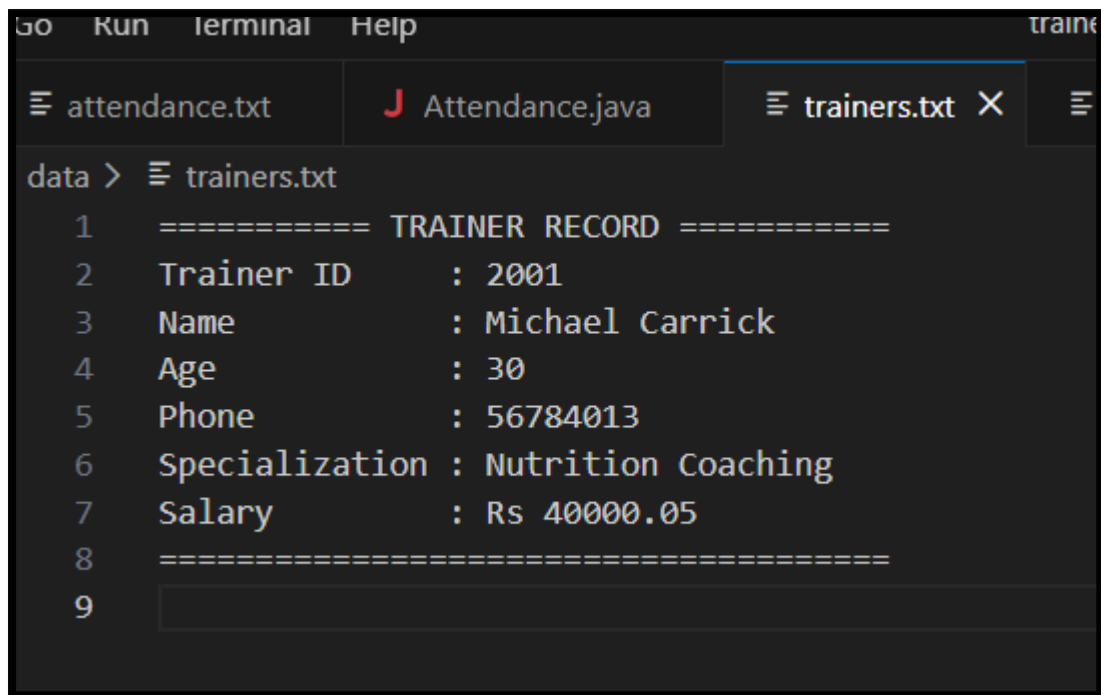
PremiumMembers.txt

```
Go Run Terminal Help PremiumMembers.txt - GymProManager
attendance.txt Attendance.java payments.txt PremiumMembers.txt X
data > PremiumMembers.txt
1  ===== PREMIUM MEMBER RECORD =====
2  Member ID           : 1002
3  Name                 : Cristiano Ronaldo
4  Age                  : 40
5  Phone                : 57894653
6  Membership Type     : Premium
7  Weight               : 78.0 kg
8  Height               : 1.86 m
9  BMI                  : 22.55
10 BMI Status          : Normal
11 Premium Benefit     : Free Personal Trainer
12 Personal Trainer Included : true
13 =====
14
```

reports.txt

```
attendance.txt Attendance.java reports.txt X payr
data > reports.txt
1  ===== MONTHLY INCOME REPORT =====
2  Report Month : 6
3  Report Year  : 2026
4  Total Income : Rs 16200.0
5  Generated By : GymPro Manager
6  =====
```

Trainers.txt



The screenshot shows an IDE window with a menu bar (Go, Run, Terminal, Help) and a toolbar. Below the toolbar are three tabs: 'attendance.txt', 'Attendance.java', and 'trainers.txt'. The 'trainers.txt' tab is active. The main editor area displays the output of a Java program, which is writing records to a file named 'trainers.txt'. The output is as follows:

```
data > trainers.txt
1  ===== TRAINER RECORD =====
2  Trainer ID      : 2001
3  Name           : Michael Carrick
4  Age            : 30
5  Phone          : 56784013
6  Specialization : Nutrition Coaching
7  Salary         : Rs 40000.05
8  =====
9
```

13.Full Java Source Codes

Main.java

```
// Import Scanner.
import java.util.Scanner;

// Main class starts the program.
public class Main {

    // Main method.
    public static void main(String[] args) {

        Scanner input = new Scanner(System.in);
        GymService gymService = new GymService(input);
        int choice = 0;

        do {
            System.out.println("\n===== GymPro Manager =====");
            System.out.println("1. Add Member");
            System.out.println("2. Add Premium Member");
            System.out.println("3. View Members");
            System.out.println("4. Update Member");
            System.out.println("5. Delete Member");
            System.out.println("6. Add Trainer");
            System.out.println("7. View Trainers");
            System.out.println("8. Process Payment");
            System.out.println("9. Record Attendance");
            System.out.println("10. View Payments");
            System.out.println("11. Monthly Income Report");
            System.out.println("12. Member BMI Statistics");
            System.out.println("13. Validation Rules");
            System.out.println("14. Search Member by ID");
            System.out.println("15. Exit");
            System.out.print("Enter your choice: ");

            if (input.hasNextInt()) {
                choice = input.nextInt();
            } else {
                input.nextLine();
                System.out.println("Invalid choice. Please enter a number.");
                continue;
            }

            switch (choice) {
                case 1:
                    gymService.addMember();
                    break;
            }
        }
    }
}
```



```

        case 2:
            gymService.addPremiumMember();
            break;
        case 3:
            gymService.viewMembers();
            break;
        case 4:
            gymService.updateMember();
            break;
        case 5:
            gymService.deleteMember();
            break;
        case 6:
            gymService.addTrainer();
            break;
        case 7:
            gymService.viewTrainers();
            break;
        case 8:
            gymService.processPayment();
            break;
        case 9:
            gymService.recordAttendance();
            break;
        case 10:
            gymService.viewPayments();
            break;
        case 11:
            gymService.monthlyIncomeReport();
            break;
        case 12:
            gymService.memberStatisticsReport();
            break;
        case 13:
            ValidationUtil.displayValidationRules();
            break;
        case 14:
            gymService.searchMemberById();
            break;
        case 15:
            System.out.println("Thank you for using GymPro Manager.");
            break;
        default:
            System.out.println("Invalid choice. Please choose between
1 and 15.");
    }

    } while (choice != 15);

```

```

        input.close();
    }
}

```

Attendance.java

```

// Import date class.
import java.time.LocalDate;

// Attendance class stores attendance records.
public class Attendance {

    private int attendanceId;
    private int memberId;
    private LocalDate date;

    // Constructor for loading saved attendance.
    public Attendance(int attendanceId, int memberId, LocalDate date) {
        this.attendanceId = attendanceId;
        this.memberId = memberId;
        this.date = date;
    }

    // Constructor for new attendance.
    public Attendance(int attendanceId, int memberId) {
        this(attendanceId, memberId, LocalDate.now());
    }

    public int getAttendanceId() {
        return attendanceId;
    }

    // Displays attendance.
    public void displayAttendance() {
        System.out.println("-----");
        System.out.println("Attendance ID: " + attendanceId);
        System.out.println("Member ID: " + memberId);
        System.out.println("Attendance Date: " + date);
    }

    // Professional text format for attendance.txt.
    public String toFileFormat() {
        return "\n===== ATTENDANCE RECORD =====\n" +
            "Attendance ID : " + attendanceId + "\n" +
            "Member ID      : " + memberId + "\n" +
            "Date           : " + date + "\n" +
            "=====";
    }
}

```

```
}  
}
```

BMIUtil.java

```
// Utility class for BMI calculation.  
public class BMIUtil {  
  
    // Calculates BMI using weight and height.  
    public static double calculateBMI(double weight, double height) {  
        return weight / (height * height);  
    }  
  
    // Returns BMI health category.  
    public static String getBMIStatus(double bmi) {  
        if (bmi < 18.5) {  
            return "Underweight";  
        } else if (bmi < 25) {  
            return "Normal";  
        } else if (bmi < 30) {  
            return "Overweight";  
        } else {  
            return "Obese";  
        }  
    }  
}
```

FileManager.java

```
// Import file classes.  
import java.io.*;  
  
// Import LocalDate.  
import java.time.LocalDate;  
  
// Import ArrayList.  
import java.util.ArrayList;  
  
// FileManager handles saving and loading files.  
public class FileManager {  
  
    // File paths.  
    public static final String MEMBER_FILE =  
        "C:/Users/DELL/Downloads/GymProManager/data/members.txt";  
  
    public static final String PREMIUM_MEMBER_FILE =  
        "C:/Users/DELL/Downloads/GymProManager/data/PremiumMembers.txt";  
}
```

```

public static final String TRAINER_FILE =
    "C:/Users/DELL/Downloads/GymProManager/data/trainers.txt";

public static final String PAYMENT_FILE =
    "C:/Users/DELL/Downloads/GymProManager/data/payments.txt";

public static final String ATTENDANCE_FILE =
    "C:/Users/DELL/Downloads/GymProManager/data/attendance.txt";

public static final String REPORT_FILE =
    "C:/Users/DELL/Downloads/GymProManager/data/reports.txt";

public static final String RECEIPT_FILE =
    "C:/Users/DELL/Downloads/GymProManager/data/receipts.txt";

// Saves data into file.
public static void saveLine(String filePath, String data) {
    try {
        FileWriter writer = new FileWriter(filePath, true);
        writer.write(data + "\n");
        writer.close();
    } catch (IOException e) {
        System.out.println("File writing error: " + e.getMessage());
    }
}

// Rewrites file.
public static void rewriteFile(String filePath, ArrayList<String> lines) {
    try {
        FileWriter writer = new FileWriter(filePath, false);

        for (String line : lines) {
            writer.write(line + "\n");
        }

        writer.close();
    } catch (IOException e) {
        System.out.println("File rewrite error: " + e.getMessage());
    }
}

// Extracts value from professional text format.
private static String getValue(String line) {
    if (line.contains(":")) {
        return line.substring(line.indexOf(":") + 1).trim();
    }
    return "";
}

```

```

    }

    // Loads members from members.txt.
    public static ArrayList<Member> loadMembers() {
        ArrayList<Member> members = new ArrayList<>();

        try {
            BufferedReader reader = new BufferedReader(new
FileReader(MEMBER_FILE));
            String line;

            int id = 0;
            String name = "";
            int age = 0;
            String phone = "";
            String type = "";
            double weight = 0;
            double height = 0;

            while ((line = reader.readLine()) != null) {
                if (line.startsWith("Member ID")) {
                    id = Integer.parseInt(getValue(line));
                } else if (line.startsWith("Name")) {
                    name = getValue(line);
                } else if (line.startsWith("Age")) {
                    age = Integer.parseInt(getValue(line));
                } else if (line.startsWith("Phone")) {
                    phone = getValue(line);
                } else if (line.startsWith("Membership Type")) {
                    type = getValue(line);
                } else if (line.startsWith("Weight")) {
                    weight = Double.parseDouble(getValue(line).replace("kg",
"".trim()));
                } else if (line.startsWith("Height")) {
                    height = Double.parseDouble(getValue(line).replace("m",
"".trim()));
                } else if (line.startsWith("===") && id != 0) {
                    members.add(new Member(id, name, age, phone, type, weight,
height));
                    id = 0;
                }
            }

            reader.close();
        } catch (Exception e) {
            System.out.println("No previous member data found.");
        }
    }

```

```

        return members;
    }

    // Loads trainers from trainers.txt.
    public static ArrayList<Trainer> loadTrainers() {
        ArrayList<Trainer> trainers = new ArrayList<>();

        try {
            BufferedReader reader = new BufferedReader(new
FileReader(TRAINER_FILE));
            String line;

            int id = 0;
            String name = "";
            int age = 0;
            String phone = "";
            String specialization = "";
            double salary = 0;

            while ((line = reader.readLine()) != null) {
                if (line.startsWith("Trainer ID")) {
                    id = Integer.parseInt(getValue(line));
                } else if (line.startsWith("Name")) {
                    name = getValue(line);
                } else if (line.startsWith("Age")) {
                    age = Integer.parseInt(getValue(line));
                } else if (line.startsWith("Phone")) {
                    phone = getValue(line);
                } else if (line.startsWith("Specialization")) {
                    specialization = getValue(line);
                } else if (line.startsWith("Salary")) {
                    salary = Double.parseDouble(getValue(line).replace("Rs",
"".trim()));
                } else if (line.startsWith("===") && id != 0) {
                    trainers.add(new Trainer(id, name, age, phone,
specialization, salary));
                    id = 0;
                }
            }

            reader.close();
        } catch (Exception e) {
            System.out.println("No previous trainer data found.");
        }

        return trainers;
    }

```

```

// Loads payments from payments.txt.
public static ArrayList<Payment> loadPayments() {
    ArrayList<Payment> payments = new ArrayList<>();

    try {
        BufferedReader reader = new BufferedReader(new
FileReader(PAYMENT_FILE));
        String line;

        int paymentId = 0;
        int memberId = 0;
        double amount = 0;
        String method = "";
        LocalDate paymentDate = LocalDate.now();
        LocalDate expiryDate = LocalDate.now();

        while ((line = reader.readLine()) != null) {
            if (line.startsWith("Payment ID")) {
                paymentId = Integer.parseInt(getValue(line));
            } else if (line.startsWith("Member ID")) {
                memberId = Integer.parseInt(getValue(line));
            } else if (line.startsWith("Amount Paid")) {
                amount = Double.parseDouble(getValue(line).replace("Rs",
"".trim()));
            } else if (line.startsWith("Payment Method")) {
                method = getValue(line);
            } else if (line.startsWith("Payment Date")) {
                paymentDate = LocalDate.parse(getValue(line));
            } else if (line.startsWith("Expiry Date")) {
                expiryDate = LocalDate.parse(getValue(line));
            } else if (line.startsWith("===") && paymentId != 0) {
                payments.add(new Payment(paymentId, memberId, amount,
method, paymentDate, expiryDate));
                paymentId = 0;
            }
        }

        reader.close();
    } catch (Exception e) {
        System.out.println("No previous payment data found.");
    }

    return payments;
}

// Loads attendance from attendance.txt.
public static ArrayList<Attendance> loadAttendance() {
    ArrayList<Attendance> attendanceList = new ArrayList<>();

```

```

        try {
            BufferedReader reader = new BufferedReader(new
FileReader(ATTENDANCE_FILE));
            String line;

            int attendanceId = 0;
            int memberId = 0;
            LocalDate date = LocalDate.now();

            while ((line = reader.readLine()) != null) {
                if (line.startsWith("Attendance ID")) {
                    attendanceId = Integer.parseInt(getValue(line));
                } else if (line.startsWith("Member ID")) {
                    memberId = Integer.parseInt(getValue(line));
                } else if (line.startsWith("Date")) {
                    date = LocalDate.parse(getValue(line));
                } else if (line.startsWith("===") && attendanceId != 0) {
                    attendanceList.add(new Attendance(attendanceId, memberId,
date));
                    attendanceId = 0;
                }
            }

            reader.close();
        } catch (Exception e) {
            System.out.println("No previous attendance data found.");
        }

        return attendanceList;
    }
}

```

GymService.java

```

// Import ArrayList.
import java.util.ArrayList;

// Import Scanner.
import java.util.Scanner;

// GymService controls all main system operations.
public class GymService {

    private ArrayList<Member> members;
    private ArrayList<Trainer> trainers;
    private ArrayList<Payment> payments;
    private ArrayList<Attendance> attendanceList;

```



```

private Scanner input;

// Loads saved data when system starts.
public GymService(Scanner input) {
    this.input = input;
    members = FileManager.loadMembers();
    trainers = FileManager.loadTrainers();
    payments = FileManager.loadPayments();
    attendanceList = FileManager.loadAttendance();
}

private int generateMemberId() {
    int max = 1000;

    for (Member member : members) {
        if (member.getId() > max) {
            max = member.getId();
        }
    }

    return max + 1;
}

private int generateTrainerId() {
    int max = 2000;

    for (Trainer trainer : trainers) {
        if (trainer.getId() > max) {
            max = trainer.getId();
        }
    }

    return max + 1;
}

private int generatePaymentId() {
    int max = 3000;

    for (Payment payment : payments) {
        if (payment.getPaymentId() > max) {
            max = payment.getPaymentId();
        }
    }

    return max + 1;
}

```

```

private int generateAttendanceId() {
    int max = 4000;

    for (Attendance attendance : attendanceList) {
        if (attendance.getAttendanceId() > max) {
            max = attendance.getAttendanceId();
        }
    }

    return max + 1;
}

private int readPositiveInt(String message) {
    int value;

    while (true) {
        System.out.print(message);

        if (input.hasNextInt()) {
            value = input.nextInt();
            input.nextLine();

            if (ValidationUtil.isPositiveInt(value)) {
                return value;
            }
        } else {
            input.nextLine();
        }

        System.out.println("Invalid input. Please enter a positive
number.");
    }
}

private double readPositiveDouble(String message) {
    double value;

    while (true) {
        System.out.print(message);

        if (input.hasNextDouble()) {
            value = input.nextDouble();
            input.nextLine();

            if (ValidationUtil.isPositiveDouble(value)) {
                return value;
            }
        } else {

```

```

        input.nextLine();
    }

    System.out.println("Invalid input. Please enter a positive decimal
number.");
    }
}

private String readText(String message) {
    String value;

    while (true) {
        System.out.print(message);
        value = input.nextLine();

        if (ValidationUtil.isValidText(value)) {
            return value;
        }

        System.out.println("Input cannot be empty.");
    }
}

private int readValidAge() {
    int age;

    while (true) {
        age = readPositiveInt("Enter Age: ");

        if (ValidationUtil.isValidAge(age)) {
            return age;
        }

        System.out.println("Invalid age. Age must be between 12 and
100.");
    }
}

private String readValidPhone() {
    String phone;

    while (true) {
        phone = readText("Enter Phone: ");

        if (ValidationUtil.isValidPhone(phone)) {
            return phone;
        }
    }
}

```

```

        System.out.println("Invalid phone. Phone must contain 7 to 10
digits.");
    }
}

private Member findMemberById(int id) {
    for (Member member : members) {
        if (member.getId() == id) {
            return member;
        }
    }

    return null;
}

private void rewriteMembersFile() {
    ArrayList<String> lines = new ArrayList<>();

    for (Member member : members) {
        lines.add(member.toFileFormat());
    }

    FileManager.rewriteFile(FileManager.MEMBER_FILE, lines);
}

public void addMember() {
    int id = generateMemberId();

    System.out.println("Generated Member ID: " + id);

    String name = readText("Enter Name: ");
    int age = readValidAge();
    String phone = readValidPhone();
    String type = readText("Enter Membership Type: ");
    double weight = readPositiveDouble("Enter Weight in kg: ");
    double height = readPositiveDouble("Enter Height in meters: ");

    Member member = new Member(id, name, age, phone, type, weight,
height);

    members.add(member);

    FileManager.saveLine(FileManager.MEMBER_FILE, member.toFileFormat());

    System.out.println("Member added successfully.");
}

public void addPremiumMember() {

```

```

        int id = generateMemberId();

        System.out.println("Generated Premium Member ID: " + id);

        String name = readText("Enter Name: ");
        int age = readValidAge();
        String phone = readValidPhone();
        String type = "Premium";
        double weight = readPositiveDouble("Enter Weight in kg: ");
        double height = readPositiveDouble("Enter Height in meters: ");
        String benefit = readText("Enter Premium Benefit: ");

        PremiumMember premiumMember =
            new PremiumMember(id, name, age, phone, type, weight, height,
benefit, true);

        members.add(premiumMember);

        FileManager.saveLine(FileManager.PREMIUM_MEMBER_FILE,
premiumMember.toFileFormat());
        FileManager.saveLine(FileManager.MEMBER_FILE,
premiumMember.toFileFormat());

        System.out.println("Premium member added successfully.");
    }

    public void viewMembers() {
        if (members.isEmpty()) {
            System.out.println("No members available.");
            return;
        }

        for (Member member : members) {
            member.displayInfo();
        }
    }

    public void searchMemberById() {
        int id = readPositiveInt("Enter Member ID to search: ");

        Member member = findMemberById(id);

        if (member == null) {
            System.out.println("Member not found.");
        } else {
            System.out.println("Member found successfully.");
            member.displayInfo();
        }
    }

```

```

    }

    public void updateMember() {
        int id = readPositiveInt("Enter Member ID to update: ");

        Member member = findMemberById(id);

        if (member == null) {
            System.out.println("Member not found.");
            return;
        }

        member.setName(readText("Enter New Name: "));
        member.setAge(readValidAge());
        member.setPhone(readValidPhone());
        member.setMembershipType(readText("Enter New Membership Type: "));
        member.setWeight(readPositiveDouble("Enter New Weight: "));
        member.setHeight(readPositiveDouble("Enter New Height: "));

        rewriteMembersFile();

        System.out.println("Member updated successfully and file rewritten.");
    }

    public void deleteMember() {
        int id = readPositiveInt("Enter Member ID to delete: ");

        Member member = findMemberById(id);

        if (member == null) {
            System.out.println("Member not found.");
            return;
        }

        members.remove(member);
        rewriteMembersFile();

        System.out.println("Member deleted successfully and file rewritten.");
    }

    public void addTrainer() {
        int id = generateTrainerId();

        System.out.println("Generated Trainer ID: " + id);

        String name = readText("Enter Trainer Name: ");
        int age = readValidAge();
        String phone = readValidPhone();
    }

```

```

        String specialization = readText("Enter Specialization: ");
        double salary = readPositiveDouble("Enter Salary: ");

        Trainer trainer = new Trainer(id, name, age, phone, specialization,
salary);

        trainers.add(trainer);

        FileManager.saveLine(FileManager.TRAINER_FILE,
trainer.toFileFormat());

        System.out.println("Trainer added successfully.");
    }

    public void viewTrainers() {
        if (trainers.isEmpty()) {
            System.out.println("No trainers available.");
            return;
        }

        for (Trainer trainer : trainers) {
            trainer.displayInfo();
        }
    }

    public void processPayment() {
        int paymentId = generatePaymentId();

        System.out.println("Generated Payment ID: " + paymentId);

        int memberId = readPositiveInt("Enter Member ID: ");
        Member member = findMemberById(memberId);

        if (member == null) {
            System.out.println("Payment failed. Member does not exist.");
            return;
        }

        int months = readPositiveInt("Enter Plan Duration in Months: ");
        double monthlyFee = readPositiveDouble("Enter Monthly Fee: ");
        String method = readText("Enter Payment Method: ");

        MembershipPlan plan = new MembershipPlan(member.getMembershipType(),
months, monthlyFee);

        double finalAmount;

        if (member instanceof PremiumMember) {

```

```

        finalAmount = plan.calculatePremiumFinalAmount();
        System.out.println("Premium Discount Applied: Rs " +
plan.calculatePremiumDiscount());
    } else {
        finalAmount = plan.calculateFinalAmount();
        System.out.println("Normal Discount Applied: Rs " +
plan.calculateDiscount());
    }

    System.out.println("-----");
    System.out.println("Payment Calculation");
    System.out.println("Total Fee: Rs " + plan.calculateTotalFee());
    System.out.println("Final Amount: Rs " + finalAmount);

    Payment payment = new Payment(paymentId, memberId, finalAmount,
method, months);

    payments.add(payment);

    FileManager.saveLine(FileManager.PAYMENT_FILE,
payment.toFileFormat());

    String receipt =
        "\n===== PAYMENT RECEIPT =====\n" +
            "Receipt ID      : " + paymentId + "\n" +
            "Member ID       : " + memberId + "\n" +
            "Member Name      : " + member.getName() + "\n" +
            "Membership Type : " + member.getMembershipType() +
"\n" +
            "Amount Paid       : Rs " + finalAmount + "\n" +
            "Payment Method    : " + method + "\n" +
            "Payment Date      : " + payment.getPaymentDate() + "\n"
+
            "Expiry Date       : " + payment.getExpiryDate() + "\n"
+
            "Status           : PAID\n" +
            "=====";

    FileManager.saveLine(FileManager.RECEIPT_FILE, receipt);

    System.out.println("Receipt generated successfully.");
    System.out.println("Membership Expiry Date: " +
payment.getExpiryDate());
    System.out.println("Payment processed successfully.");
}

public void recordAttendance() {
    int attendanceId = generateAttendanceId();

```



```

        System.out.println("Generated Attendance ID: " + attendanceId);

        int memberId = readPositiveInt("Enter Member ID: ");

        Member member = findMemberById(memberId);

        if (member == null) {
            System.out.println("Attendance failed. Member does not exist.");
            return;
        }

        Attendance attendance = new Attendance(attendanceId, memberId);

        attendanceList.add(attendance);

        FileManager.saveLine(FileManager.ATTENDANCE_FILE,
attendance.toFileFormat());

        System.out.println("Attendance recorded successfully.");
    }

    public void viewPayments() {
        if (payments.isEmpty()) {
            System.out.println("No payments available.");
            return;
        }

        for (Payment payment : payments) {
            payment.displayPayment();
        }
    }

    public void monthlyIncomeReport() {
        int month = readPositiveInt("Enter month number, example 1 for
January: ");
        int year = readPositiveInt("Enter year, example 2026: ");

        ReportGenerator.displayMonthlyIncomeReport(payments, month, year);

        double total = ReportGenerator.calculateMonthlyIncome(payments, month,
year);

        String report =
            "\n===== MONTHLY INCOME REPORT =====\n" +
            "Report Month : " + month + "\n" +
            "Report Year  : " + year + "\n" +
            "Total Income : Rs " + total + "\n" +

```

```

        "Generated By : GymPro Manager\n" +
        "=====";

    FileManager.saveLine(FileManager.REPORT_FILE, report);
}

public void memberStatisticsReport() {
    ReportGenerator.displayMemberStatistics(members);
}
}

```

Member.java

```

// Member class represents a normal gym member.
public class Member extends Person {

    // Member-specific attributes.
    private String membershipType;
    private double weight;
    private double height;

    // Constructor.
    public Member(int id, String name, int age, String phone, String
membershipType, double weight, double height) {
        super(id, name, age, phone);
        this.membershipType = membershipType;
        this.weight = weight;
        this.height = height;
    }

    // Getter methods.
    public String getMembershipType() {
        return membershipType;
    }

    public double getWeight() {
        return weight;
    }

    public double getHeight() {
        return height;
    }

    // Setter methods.
    public void setMembershipType(String membershipType) {
        this.membershipType = membershipType;
    }
}

```

```

public void setWeight(double weight) {
    this.weight = weight;
}

public void setHeight(double height) {
    this.height = height;
}

// Calculates BMI.
public double calculateBMI() {
    return BMIUtil.calculateBMI(weight, height);
}

// Gets BMI status.
public String getBMIStatus() {
    return BMIUtil.getBMIStatus(calculateBMI());
}

// Displays member details.
@Override
public void displayInfo() {
    System.out.println("-----");
    System.out.println("Member ID: " + getId());
    System.out.println("Name: " + getName());
    System.out.println("Age: " + getAge());
    System.out.println("Phone: " + getPhone());
    System.out.println("Membership Type: " + membershipType);
    System.out.println("Weight: " + weight + " kg");
    System.out.println("Height: " + height + " m");
    System.out.printf("BMI: %.2f\n", calculateBMI());
    System.out.println("BMI Status: " + getBMIStatus());
}

// Professional text format for members.txt.
public String toFileFormat() {
    return "\n===== MEMBER RECORD =====\n" +
        "Member ID      : " + getId() + "\n" +
        "Name             : " + getName() + "\n" +
        "Age              : " + getAge() + "\n" +
        "Phone            : " + getPhone() + "\n" +
        "Membership Type  : " + membershipType + "\n" +
        "Weight           : " + weight + " kg\n" +
        "Height           : " + height + " m\n" +
        "BMI              : " + String.format("%.2f", calculateBMI()) +
"\n" +
        "BMI Status       : " + getBMIStatus() + "\n" +
        "=====";
}

```

```
}
```

MembershipPlan.java

```
// MembershipPlan handles membership payment calculations.
public class MembershipPlan {

    // Plan details.
    private String planName;
    private int durationMonths;
    private double monthlyFee;

    // Constructor.
    public MembershipPlan(String planName, int durationMonths, double
monthlyFee) {
        this.planName = planName;
        this.durationMonths = durationMonths;
        this.monthlyFee = monthlyFee;
    }

    // Calculates total fee before discount.
    public double calculateTotalFee() {
        return monthlyFee * durationMonths;
    }

    // Calculates normal discount.
    public double calculateDiscount() {
        if (durationMonths >= 12) {
            return calculateTotalFee() * 0.20;
        } else if (durationMonths >= 6) {
            return calculateTotalFee() * 0.10;
        } else {
            return 0;
        }
    }

    // Calculates premium discount.
    public double calculatePremiumDiscount() {
        return calculateTotalFee() * 0.25;
    }

    // Calculates normal final amount.
    public double calculateFinalAmount() {
        return calculateTotalFee() - calculateDiscount();
    }

    // Calculates premium final amount.
    public double calculatePremiumFinalAmount() {
```

```

        return calculateTotalFee() - calculatePremiumDiscount();
    }
}

```

Payment.java

```

// Import date class.
import java.time.LocalDate;

// Payment class stores payment transaction details.
public class Payment {

    private int paymentId;
    private int memberId;
    private double amount;
    private String paymentMethod;
    private LocalDate paymentDate;
    private LocalDate expiryDate;

    // Constructor for loading old records.
    public Payment(int paymentId, int memberId, double amount, String
paymentMethod, LocalDate paymentDate) {
        this.paymentId = paymentId;
        this.memberId = memberId;
        this.amount = amount;
        this.paymentMethod = paymentMethod;
        this.paymentDate = paymentDate;
        this.expiryDate = paymentDate;
    }

    // Constructor for loading records with expiry date.
    public Payment(int paymentId, int memberId, double amount, String
paymentMethod,
                    LocalDate paymentDate, LocalDate expiryDate) {
        this.paymentId = paymentId;
        this.memberId = memberId;
        this.amount = amount;
        this.paymentMethod = paymentMethod;
        this.paymentDate = paymentDate;
        this.expiryDate = expiryDate;
    }

    // Constructor for new payment.
    public Payment(int paymentId, int memberId, double amount, String
paymentMethod, int durationMonths) {
        this.paymentId = paymentId;
        this.memberId = memberId;
        this.amount = amount;
    }
}

```

```

        this.paymentMethod = paymentMethod;
        this.paymentDate = LocalDate.now();
        this.expiryDate = paymentDate.plusMonths(durationMonths);
    }

    public int getPaymentId() {
        return paymentId;
    }

    public double getAmount() {
        return amount;
    }

    public LocalDate getPaymentDate() {
        return paymentDate;
    }

    public LocalDate getExpiryDate() {
        return expiryDate;
    }

    // Displays payment in console.
    public void displayPayment() {
        System.out.println("-----");
        System.out.println("Payment ID: " + paymentId);
        System.out.println("Member ID: " + memberId);
        System.out.println("Amount Paid: Rs " + amount);
        System.out.println("Payment Method: " + paymentMethod);
        System.out.println("Payment Date: " + paymentDate);
        System.out.println("Expiry Date: " + expiryDate);
    }

    // Professional text format for payments.txt.
    public String toFileFormat() {
        return "\n===== PAYMENT RECORD =====\n" +
            "Payment ID      : " + paymentId + "\n" +
            "Member ID       : " + memberId + "\n" +
            "Amount Paid      : Rs " + amount + "\n" +
            "Payment Method   : " + paymentMethod + "\n" +
            "Payment Date     : " + paymentDate + "\n" +
            "Expiry Date      : " + expiryDate + "\n" +
            "=====";
    }
}

```

Person.java

```
// Abstract parent class for common person information.
public abstract class Person {

    // Common attributes shared by Member and Trainer.
    private int id;
    private String name;
    private int age;
    private String phone;

    // Constructor to initialize common details.
    public Person(int id, String name, int age, String phone) {
        this.id = id;
        this.name = name;
        this.age = age;
        this.phone = phone;
    }

    // Getter methods.
    public int getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public int getAge() {
        return age;
    }

    public String getPhone() {
        return phone;
    }

    // Setter methods.
    public void setName(String name) {
        this.name = name;
    }

    public void setAge(int age) {
        this.age = age;
    }

    public void setPhone(String phone) {
        this.phone = phone;
    }
}
```

```

// Abstract method used for polymorphism.
public abstract void displayInfo();
}

```

PremiumMember.java

```

// PremiumMember is a specialized child class of Member.
public class PremiumMember extends Member {

    // Extra premium member attributes.
    private String premiumBenefit;
    private boolean personalTrainerIncluded;

    // Constructor.
    public PremiumMember(int id, String name, int age, String phone, String
membershipType,
                        double weight, double height, String premiumBenefit,
                        boolean personalTrainerIncluded) {

        super(id, name, age, phone, membershipType, weight, height);
        this.premiumBenefit = premiumBenefit;
        this.personalTrainerIncluded = personalTrainerIncluded;
    }

    // Displays premium member details.
    @Override
    public void displayInfo() {
        super.displayInfo();
        System.out.println("Premium Benefit: " + premiumBenefit);
        System.out.println("Personal Trainer Included: " +
personalTrainerIncluded);
    }

    // Professional text format for PremiumMembers.txt.
    @Override
    public String toFileFormat() {
        return "\n===== PREMIUM MEMBER RECORD =====\n" +
            "Member ID           : " + getId() + "\n" +
            "Name                   : " + getName() + "\n" +
            "Age                     : " + getAge() + "\n" +
            "Phone                  : " + getPhone() + "\n" +
            "Membership Type        : " + getMembershipType() + "\n" +
            "Weight                  : " + getWeight() + " kg\n" +
            "Height                  : " + getHeight() + " m\n" +
            "BMI                     : " + String.format("%.2f",
calculateBMI()) + "\n" +
            "BMI Status              : " + getBMIStatus() + "\n" +
            "Premium Benefit         : " + premiumBenefit + "\n" +

```



```

        "Personal Trainer Included : " + personalTrainerIncluded +
"\n" +
        "=====";
    }
}

```

ReportGenerator.java

```

// Import ArrayList.
import java.util.ArrayList;

// Import LocalDate.
import java.time.LocalDate;

// ReportGenerator handles reports and statistics.
public class ReportGenerator {

    // Calculates monthly income.
    public static double calculateMonthlyIncome(ArrayList<Payment> payments,
int month, int year) {
        double totalIncome = 0;

        for (Payment payment : payments) {
            LocalDate date = payment.getPaymentDate();

            if (date.getMonthValue() == month && date.getYear() == year) {
                totalIncome += payment.getAmount();
            }
        }

        return totalIncome;
    }

    // Counts monthly payments.
    public static int countMonthlyPayments(ArrayList<Payment> payments, int
month, int year) {
        int count = 0;

        for (Payment payment : payments) {
            LocalDate date = payment.getPaymentDate();

            if (date.getMonthValue() == month && date.getYear() == year) {
                count++;
            }
        }

        return count;
    }
}

```

```

// Displays monthly report.
public static void displayMonthlyIncomeReport(ArrayList<Payment> payments,
int month, int year) {
    double totalIncome = calculateMonthlyIncome(payments, month, year);
    int count = countMonthlyPayments(payments, month, year);

    System.out.println("-----");
    System.out.println("MONTHLY INCOME REPORT");
    System.out.println("Month: " + month);
    System.out.println("Year: " + year);
    System.out.println("Number of Payments: " + count);
    System.out.println("Total Income: Rs " + totalIncome);
}

// Displays BMI statistics.
public static void displayMemberStatistics(ArrayList<Member> members) {
    int underweight = 0;
    int normal = 0;
    int overweight = 0;
    int obese = 0;

    for (Member member : members) {
        String status = member.getBMIStatus();

        if (status.equalsIgnoreCase("Underweight")) {
            underweight++;
        } else if (status.equalsIgnoreCase("Normal")) {
            normal++;
        } else if (status.equalsIgnoreCase("Overweight")) {
            overweight++;
        } else {
            obese++;
        }
    }

    System.out.println("-----");
    System.out.println("MEMBER BMI STATISTICS");
    System.out.println("Total Members: " + members.size());
    System.out.println("Underweight Members: " + underweight);
    System.out.println("Normal Members: " + normal);
    System.out.println("Overweight Members: " + overweight);
    System.out.println("Obese Members: " + obese);
}
}

```

Trainer.java

```
// Trainer class represents a gym trainer.
public class Trainer extends Person {

    // Trainer-specific attributes.
    private String specialization;
    private double salary;

    // Constructor.
    public Trainer(int id, String name, int age, String phone, String
specialization, double salary) {
        super(id, name, age, phone);
        this.specialization = specialization;
        this.salary = salary;
    }

    // Displays trainer details.
    @Override
    public void displayInfo() {
        System.out.println("-----");
        System.out.println("Trainer ID: " + getId());
        System.out.println("Name: " + getName());
        System.out.println("Age: " + getAge());
        System.out.println("Phone: " + getPhone());
        System.out.println("Specialization: " + specialization);
        System.out.println("Salary: Rs " + salary);
    }

    // Professional text format for trainers.txt.
    public String toFileFormat() {
        return "\n===== TRAINER RECORD =====\n" +
            "Trainer ID      : " + getId() + "\n" +
            "Name            : " + getName() + "\n" +
            "Age             : " + getAge() + "\n" +
            "Phone           : " + getPhone() + "\n" +
            "Specialization : " + specialization + "\n" +
            "Salary          : Rs " + salary + "\n" +
            "=====";
    }
}
```

ValidationUtil.java

```
// Utility class for input validation.
public class ValidationUtil {

    // Checks that text is not empty.
```

```

public static boolean isValidText(String text) {
    return text != null && !text.trim().isEmpty();
}

// Checks positive integer.
public static boolean isPositiveInt(int number) {
    return number > 0;
}

// Checks positive decimal number.
public static boolean isPositiveDouble(double number) {
    return number > 0;
}

// Checks valid age range.
public static boolean isValidAge(int age) {
    return age >= 12 && age <= 100;
}

// Checks phone number contains 7 to 10 digits.
public static boolean isValidPhone(String phone) {
    return phone != null && phone.matches("\\d{7,10}");
}

// Displays validation rules.
public static void displayValidationRules() {
    System.out.println("-----");
    System.out.println("Validation Rules");
    System.out.println("Name cannot be empty.");
    System.out.println("Age must be between 12 and 100.");
    System.out.println("Phone must contain 7 to 10 digits.");
    System.out.println("Numbers must be positive.");
}
}

```

REFERENCES

Website:

draw.io. (n.d.) UML Use Case Diagrams. Available at: <https://www.drawio.com/docs/diagram-types/uml/use-case-diagrams/> (Accessed: 8 May 2026).

Website:

Oracle Corporation. (2026) Java FileWriter Class Documentation. Available at: <https://docs.oracle.com/en/java/javase/26/docs/api/java.base/java/io/FileWriter.html> (Accessed: 14 May 2026).

Website:

W3Schools. (2026) Java Tutorial. Available at: <https://www.w3schools.com/java/> (Accessed: 17 May 2026).

Website:

PlantUML. (2026) Class Diagram Documentation. Available at: <https://plantuml.com/class-diagram> (Accessed: 19 May 2026).

Video:

Random Code. (2021) UML Basics (with PlantUML) #1: Class Diagram – Basic Elements. YouTube. Available at: <https://www.youtube.com/watch?v=DICgVziU7z8> (Accessed: 21 May 2026).

Website:

GeeksforGeeks. (2026) Java Collections Framework. Available at: <https://www.geeksforgeeks.org/collections-in-java-2/> (Accessed: 24 May 2026).

AI Tool:

OpenAI. (2026) ChatGPT (GPT-5.5) [Large language model]. Available at: <https://chatgpt.com> (Accessed: 26 May 2026).

Video:

Neso Academy. (2020) UML Class Diagram. YouTube. Available at: <https://www.youtube.com/watch?v=ksvwF2sUjHM> (Accessed: 28 May 2026).

Video:

Derek Banas. (n.d.) UML 2.0 Tutorial. YouTube. Available at: <https://www.youtube.com/watch?v=WnMQ8HlmeXc> (Accessed: 1 June 2026).

Video:

Programming with Mosh. (2023) Object-Oriented Programming in Java. YouTube. Available at: https://www.youtube.com/watch?v=6T_HgnjoYwM (Accessed: 3 June 2026).